

第3章 继承与派生

面向对象的大型程序设计最重要的技术之一就是代码重用性 (software reusability)。C++语言提供了类的继承 (inheritance) 机制，极大地提高了代码重用性。本章将重点介绍 C++ 提供的继承与派生技术。C++ 在面向对象程序设计中，引入继承机制，使得程序的设计更符合发展规律，即事物的发展是一个从低级到高级的发展过程，类的继承也是反映由原始的简单代码到丰富的高级代码的过程。它能帮助我们描述事物的层次关系，有效而精确的理解事物，理解事物直到本质。

3.1 继承与派生的概念

前面章节中学习过：类是对某一类事物的属性抽象和描述。例如，可以采用 student 类对“学生”进行描述，可以将其抽象为“学号、姓名、性别、...”等属性。然而，在自然界法则中，许多相关事物之间具有一定层次关系。这种层次关系在事物的属性上体现了共性和个性的关系。例如，如图 3-1 所示。“小学生”或“中学生”与“学生”就有一定的层次关系。“学生”属性体现“小学生”和“中学生”的共同属性。而“小学生”和“中学生”还具备自己的个体属性。

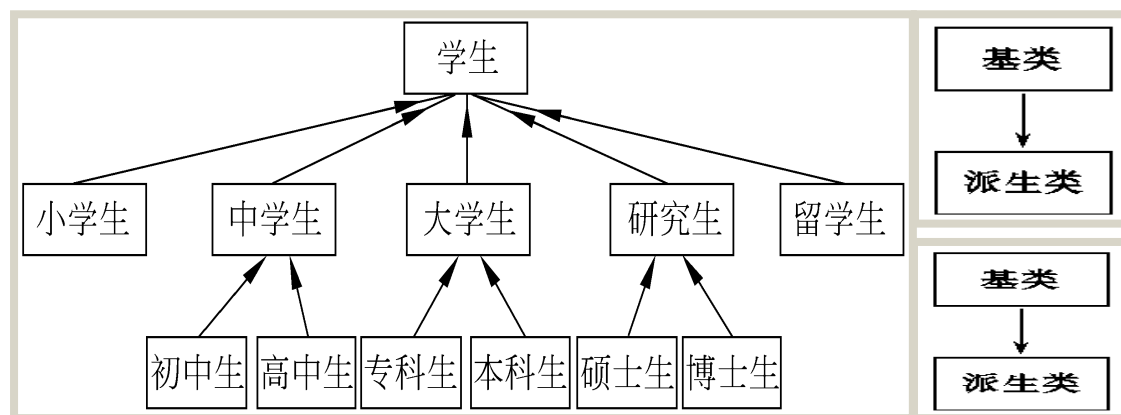


图3-1 自然界事物层次关系实例

一个类中包含了若干数据成员和成员函数。在不同的类中，数据成员和成员函数是不相同的。但有时两个类的内容基本相同或部分相同。假设学生类 student 中包含了若干数据成员和成员函数。中学生类 middle_school_student 中包含了类 student 部分的相同成员。如何高效地申明新的类 middle_school_student 呢？这就要使用 C++ 提供的继承机制。

在 C++ 中，所谓“继承”就是在一个已有类的基础上建立一个新类。已有类（例如“学生”）称为“基类 (base class)”或“父类 (father class)”。新建类（例如“中学生”等）称为“派生类 (derived class)”或“子类 (son class)”。

因此，继承与派生是从 2 个不同的视角来描述同一的事件。一个新类从已有的类那里获得其已有特性，这种现象称为类的继承。通过继承，一个新建子类从已有的父类那里获得父类的属性。从另一角度说，从已有的类（父类）产生一个新的子类，称为类的派生。派生类继承了基类的所有数据成员和成员函数，并可以对成员作必要的增加或调整。关于基类和派生类的关系，可以表述为：派生类是基类的具体化，而基类则是派生类的抽象。

另外，一个基类可以派生出多个派生类，每一个派生类又可以作为基类再派生出新的派生类，例如，“学生”类作为基类，派生“中学生”类；然后，也可以通过“中学生”类作为基类，派生“高中生”类。因此基类和派生类是相对而言的。如图 3-1 所示。在该图中，介绍的是最简单的情况：一个派生类只从一个基类派生，这称为单继承(single inheritance)，这种继承关系所形成的层次是一个树形结构。在本书中约定，箭头表示继承的方向，从派生类指向基类。

一个派生类不仅可以从一个基类派生，也可以从多个基类派生。一个派生类有两个或多个基类的称为多重继承(multiple inheritance)，这种继承关系所形成的结构如图 3-2 所示。

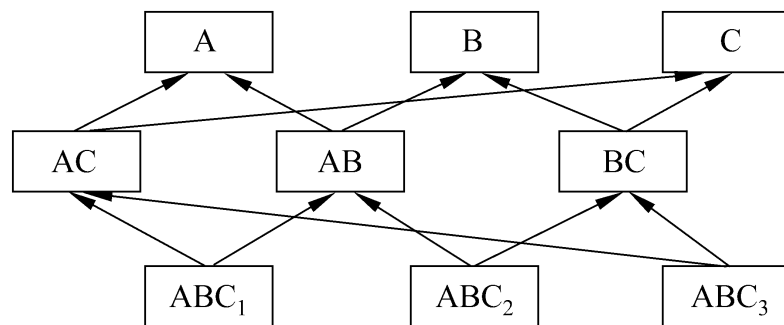


图 3-2 多重继承示意图

3.2 派生类的声明方式及其组成

3.2.1 派生类的声明方式

如果在已有基类的基础上，通过继承机制，派生一个新的派生类。则这个新的派生类申明一般格式如下。

```
class 派生类名: [继承方式] 基类名
{派生类新增加的成员};
```

其中：继承方式包括：public（公用的）、private（私有的）和 protected（受保护的）。此项是可选的，缺省则默认为 private（私有的）。

例如：假设已经声明了一个基类 Student

在此基础上，通过单继承建立一个派生类 Primary_Student。

```
class Primary_Student: public Student //声明基类是 Student
```

```

{public:
void display_1( )    //新增加的成员函数
{cout<<" age: " <<age<<endl;
 cout<<" address: " <<addr<<endl;}
private:
int age;           //新增加的数据成员
string addr;       //新增加的数据成员
};

```

基类名前面有 public 的称为“公用继承(public inheritance)”。

3.2.2 派生类的组成

派生类Primary_Student 中的成员包括从基类Student 继承过来的成员和自己增加的成员两大部分。在基类中包括数据成员和成员函数两部分。因此，派生类分为两大部分：一部分是从基类继承来的成员，另一部分是在声明派生类时增加的部分。每一部分均分别包括数据成员和成员函数。派生类的构成以及与基类的成员关系如图 3-3 所示。

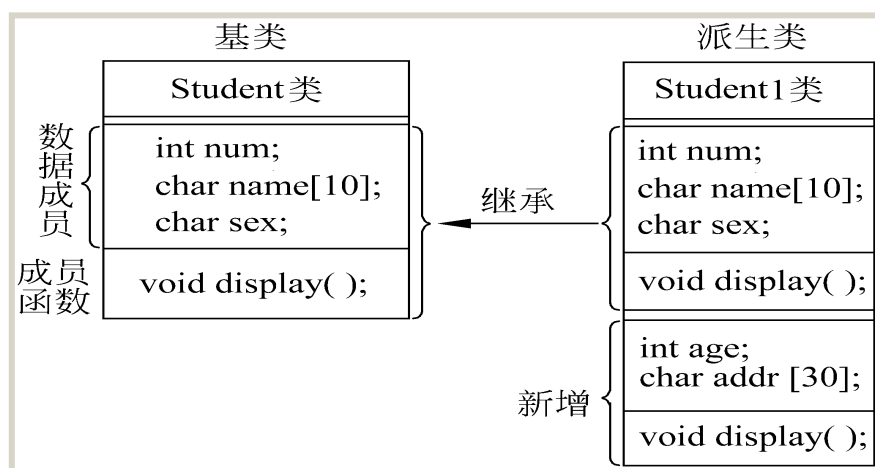


图 3-3 派生类的构成以及与基类的成员关系

从上述实例中可以看出，派生类的成员构成包括以下 2 部分。

(1) 从基类接收成员。

派生类把基类全部的成员(不包括构造函数和析构函数)接收过来，也就是说是没有选择的，不能选择接收其中一部分成员，而舍弃另一部分成员。因此，基类选择就特别重要，力求使得冗余量最小。

但是，需要特别注意，在声明派生类时，一般还应当自己定义派生类的构造函数和析构函数，因为构造函数和析构函数是不能从基类继承的。

另外，派生类接收基类成员是程序人员不能选择的，但是程序人员可以对这些成员的访问属性作某些调整。使得有些成员不可见、或者在类内不可访问等。有关这部分的详细内容，

在继承方式的学习中再具体介绍。

(2) 在声明派生类时增加的成员。

用户可以根据新的派生类的需求,新增一些类成员,体现派生类对基类功能的扩展。因此,一般做法是:先声明一个基类,在此基类中只提供某些最基本的功能,而另外有些功能并未实现,然后在声明派生类时加入某些具体的功能,形成适用于某一特定应用的派生类。可以说:派生类是基类定义的延续。通过对基类声明的延续,将一个抽象的基类转化成具体的派生类。

第1点体现了基类“遗传”特性,第2点体现了派生类的“变异”特性。大自然就是在这“遗传”和“变异”中,繁衍出大千世界的形形色色物种。

3.3 派生类成员的访问属性与继承方式

从3.2节中,派生类申明一般格式可以看出,派生类从基类继承的方式有3种,分别是:public(公用的)、private(私有的)和protected(受保护的)。这3种不同的继承方式决定着派生类中成员的关系和访问属性。

既然派生类中包含基类成员和派生类自己新增的成员,这就产生了这两部分成员的关系及其访问属性的问题。具体包括2种情况:第1种情况,派生类的成员函数访问基类的成员。

第2种情况,在派生类外访问基类的成员。

基类成员在派生类中的访问属性取决2方面因素:派生类的继承方式,以及基类成员所声明的访问属性。具体情况可以归纳为如下3种情况。

(1) 公用继承(public inheritance)

基类的公用成员和保护成员在派生类中保持原有访问属性。其私有成员仍为基类私有。在派生类中不可访问。

(2) 私有继承(private inheritance)

基类的公用成员和保护成员在派生类中成了私有成员。其私有成员仍为基类私有,在派生类中不可访问。

(3) 受保护的继承(protected inheritance)

基类的公用成员和保护成员在派生类中成了保护成员,其私有成员仍为基类私有,在派生类中不可访问。

在此需要补充说明:前面只是介绍了类中成员访问属性private和public。现在介绍第3种类成员访问属性,即protected。保护成员的访问属性规则是:不能被外界引用,但可以被派生类的成员引用,这一点和private相同,但是,从上述3种继承方式来看,protected与private成员访问属性在派生类中的访问属性是不一样的。这也是引入protected成员的目的所在。希望同学们能深入领会。

表3.1概括了上述3种继承方式在派生类中对基类成员的访问属性的改变。基本规则是:基类的私有成员在派生类,无论是何种继承方式都是“无可访问”。而“public”继承没有

改变访问属性，“protected” 继承后的基类访问属性在派生类都改变为 “protected”，
“private” 继承后的基类访问属性在派生类都改变为 “private”。

表 3.1 3 种继承方式在派生类中对基类成员的访问属性的变化

基类成员访问控制	继承访问控制	在派生类中的访问控制
public	public	public
protected		protected
private		不可访问
public	protected	protected
protected		protected
private		不可访问
public	private	private
protected		private
private		不可访问

下面结合实例分别深入讨论这 3 种情况。

3.3.1 公用继承方式

所谓公用继承，是指在定义一个派生类时将基类的继承方式指定为 public。用公用继承方式建立的派生类称为公用派生类(public derived class)，其基类称为公用基类(public base class)。

采用公用继承方式时，基类的公用成员和保护成员在派生类中仍然保持其公用成员和保护成员的属性，而基类的私有成员在派生类中并没有成为派生类的私有成员，它仍然是基类的私有成员。也就是说：在派生类中，只有从基类继承来的成员函数可以引用它，而不能被派生类的新增成员函数引用，因此就成为派生类中的不可访问的成员。公用基类的成员在派生类中的访问属性见书中表 3-1。表 3-1 公用基类在派生类中的访问属性

公用基类的成员	在公用派生类中的访问属性
私有成员	不可访问
公用成员	公用
保护成员	保护

实例 3-1 公有继承方式。分析下列程序，领会派生类的访问属性。

```
#include <iostream>
#include <string>
using namespace std;
```

```

class Employee
{
public : //基类公用成员
    void get_value( )
    {
        cin>>num>>name>>sex;
    }
    void show( )
    {
        cout<<" num: "<<num<<endl;
        cout<<" name: "<<name<<endl;
        cout<<" sex: "<<sex<<endl;
    }
private : //基类私有成员
    int num;
protected : //基类保护成员
    string name;
public :
    char sex;
};

class Intern: public Employee //以public方式声明派生类
{
public:
    void display( ) //派生类新增成员函数
    {
        //cout<<" num: "<<num<<endl; //引用基类的私有成员, 错误
        cout<<" name: "<<name<<endl; //引用基类的保护成员, 正确
        cout<<" sex: "<<sex<<endl; //引用基类的公用成员, 正确
        cout<<" age: "<<age<<endl; //引用派生类的新增私有成员, 正确
        cout<<" address: "<<addr<<endl; //引用派生类的新增公用成员, 正确
    }
private:
    int age;
public:
    string addr;
};

int main( )
{
    Intern stud;//定义派生类的对象stud
    //stud. num=12; //在类外引用基类的私有成员, 错误
    //stud. name = " yfh" ; //在类外引用基类的保护成员, 错误
    stud.show( );//在类外引用基类的公用成员函数, 正确
}

```

```

stud.display();//在类外引用派生类公用成员函数，正确
return 0;
}

```

由于基类的私有成员对派生类来说是不可访问的，因此在派生类的 `display()` 函数中直接引用基类的私有数据成员 `num` 是不允许的，只能通过基类的公用成员函数 `show()` 来引用基类的私有数据成员。但直接引用基类的保护数据成员 `name` 和公用数据成员 `sex` 是允许的。

因此，在 `main` 函数中可以分别调用基类的 `show` 函数和派生类中的 `display` 函数，先后输出 5 个数据。

另外，还可以在派生类的 `display` 函数中调用基类的 `show` 函数。这样在主函数中只要写一行：`stud.display()`；即可输出 5 个数据。

3.3.2 私有继承方式

所谓私有继承方式，是指在声明一个派生类时将基类的继承方式指定为 `private` 的。用私有继承方式建立的派生类称为私有派生类(private derived class)，其基类称为私有基类(private base class)。私有继承时，基类的公用成员和保护成员在派生类中的访问属性相当于派生类中的私有成员，即派生类的成员函数能访问它们，而在派生类外不能访问它们。而基类的私有成员在派生类中对于派生类新增成员不可访问，但派生类中从基类继承来的成员函数可以引用它们。私有继承方式的基类成员在派生类中的访问属性如表 3-2 所示。

表 3-2 私有继承方式的基类成员在派生类中的访问属性

私有基类的成员	在私有派生类中的访问属性
私有成员	不可访问
公用成员	私有
保护成员	私有

派生类的私有继承方式表示将基类原来能被外界引用的成员隐藏起来，不让外界引用。因此，基类的公用成员和保护成员理所当然地成为派生类中的私有成员。而基类的私有成员按规定只能被基类的成员函数引用，在基类之外当然不能访问他们。因此，它们在派生类中是隐蔽的，不可访问的。根据私有继承方式的特点，对于不需要再往下继承的类的功能可以用私有继承方式把它隐蔽起来，这样，下一层的派生类无法访问它的任何成员。

实例 3-2 将实例 3-1 中的公用继承方式改为用私有继承方式(基类 `Employee` 不改)。分析派生类的成员访问属性。

```

#include <iostream>
#include <string>

```

```

using namespace std;

class Employee
{
public : //基类公用成员
    void get_value( )
    {
        cin>>num>>name>>sex;
    }
    void show( )
    {
        cout<<" num: "<<num<<endl;
        cout<<" name: "<<name<<endl;
        cout<<" sex: "<<sex<<endl;
    }
private : //基类私有成员
    int num;
protected : //基类保护成员
    string name;
public :
    char sex;
};

class Intern: private Employee //以public方式声明派生类
{
public:
    void display( )           //派生类新增成员函数
    {
        //cout<<" num: "<<num<<endl; //引用基类的私有成员, 错误
        cout<<" name: "<<name<<endl; //引用基类的保护成员, 正确
        cout<<" sex: "<<sex<<endl;    //引用基类的公用成员, 正确
        cout<<"age: "<<age<<endl;    //引用派生类的私有成员, 正确
        cout<<"address: "<<addr<<endl; //引用派生类的私有成员, 正确
    }
private:
    int age;
public:
    string addr;
};

int main( )
{
    Intern stud1;//定义派生类的对象stud
    //stud1. num=12;          //在类外引用基类的私有成员, 错误

```



```

//stud1. name =" yfh" ; //在类外引用基类的保护成员，错误
//stud1.show(); //错误，私有基类公用成员在派生类中是私有
stud1.display( ); //正确。Display在Intern类是公有
//stud1.age=18; //错误。外界不能引用派生类的私有成员
return 0;
}

```

通过分析上述程序，可以进一步熟悉私有继承方式的访问规则。

(1) 不能通过派生类对象(如 stud1) 引用从私有基类继承过来的任何成员(如 stud1.show() 或 stud1.num。

(2) 派生类的成员函数不能访问私有基类的私有成员，但可以访问私有基类的公用成员和保护成员。

3.3.3 保护成员和保护继承

1. 类的保护成员

在前面介绍类的申明方法时，只是重点介绍 2 种成员访问属性。其实还有第 3 种成员访问属性，即 protected(保护)。类中的成员访问属性申明为“受保护”，则简称“保护成员”。

从类成员的访问属性角度来看，保护成员等价于私有成员。只能在类内访问，不能在类外访问(通过对象来访问)。

但从派生类的继承属性角度来看，保护成员完全不同于私有成员。因为基类的保护成员可以被派生类的成员函数引用。但基类的私有成员，在任何派生类中都是不能访问的(不可见)。

2. 保护继承方式

所谓保护继承方式，是指在定义一个派生类时将基类的继承方式指定为 protected 的。用保护继承方式建立的派生类称为保护派生类(protected derived class)。其基类称为保护基类(protected base class)。

保护继承的特点是：保护基类的公用成员和保护成员在派生类中都成了保护成员，其私有成员仍为基类私有，在派生类中变成不可见(不可访问)。也就是把基类原有的公用成员也保护起来，只能在派生类内访问，不让类外通过对象来访问。

比较一下私有继承和保护继承(也就是比较在私有派生类中和在保护派生类中的访问属性)，可以发现，在直接派生类中，以上两种继承方式的作用实际上是相同的，即在类外不能访问任何成员，而在派生类中可以通过成员函数访问基类中的公用成员和保护成员。但是如果继续派生，在新的派生类中，两种继承方式的作用就不同了。例如，如果以公用继承方式派生出一个新派生类，原来私有基类中的成员在新派生类中都成为不可访问的成员，无论在派生类内或类外都不能访问，而原来保护基类中的公用成员和保护成员在新派生类中为保护成员，可以被新派生类的成员函数访问。

如果善于利用保护成员，可以在类的层次结构中找到数据共享与成员隐蔽之间的结合点。既可实现某些成员的隐蔽，又可方便地继承，还能实现代码重用与扩充。

表 3-3 保护继承方式的基类成员在派生类中的访问属性

保护基类的成员	在保护派生类中的访问属性
私有成员	不可访问
公用成员	保护
保护成员	保护

下面通过一个例子说明保护成员和保护继承方式的应用方法。

实例 3-3 基类的保护成员和派生类的保护继承的应用方式举例。

```
#include <iostream>
#include <string>
using namespace std;

class Employee
{
public : //基类公用成员
    void show( )
    {
        cout<<" num: "<<num<<endl;
        cout<<" name: "<<name<<endl;
        cout<<" sex: "<<sex<<endl;
    }
protected : //基类保护成员
    void show( );
    int num;
    string name;
private:
    char sex;
};

class Intern: protected Employee //以public方式声明派生类
{
public:
    void display( ) //派生类新增成员函数
    {
        cout<<"num: "<<num<<endl; //引用基类的保护成员，合法
        cout<<"name: "<<name<<endl; //引用基类的保护成员，合法
        //cout<<"sex: "<<sex<<endl; //引用基类的私有成员，错误
        cout<<"age: "<<age<<endl; //引用派生类的私有成员，合法
        cout<<"address: "<<addr<<endl; //引用派生类的私有成员，合法
    }
};
```

```

    }
private:
    int age;
    string addr;
};

int main( )
{
    Intern stud1; //定义派生类的对象stud
    stud1.display( ); //类外访问派生类中的公用成员函数，合法
    //stud1.show( ); //外界不能访问基类的保护成员，错误
    return 0;
}

```

通过分析程序实例 3-3，可以得出如下结论。

(1) 基类的保护成员和私有成员不同之处在于：把保护成员的访问范围扩展到派生类中。因此，在派生类的成员函数中引用基类的保护成员 num 和 name 是合法的，引用基类的私有成员 sex 是非法的。

(2) 基类的保护成员和公用成员不同之处在于：通过公用继承方式时不能通过派生类对象名去访问基类的保护成员，但可以访问基类的公用成员。

综上所述，通过对上述 3 种继承方式的学习，可以总结如下规则。基类的成员只有 3 种访问属性，但在派生类中的成员其实有 4 种不同的访问属性

- (1) public(派生类内和派生类外都可以访问);
- (2) protected(派生类内可以访问，派生类外不能访问，其下一层的派生类可以访问);
- (3) private(派生类内可以访问，派生类外不能访问);
- (4) 不可见(派生类内和派生类外都不能访问)。

3.3.4 使用 using 申明来改变基类成员在派生类中访问属性

按照上面介绍派生类的 3 种继承方式，基类 3 种不同访问属性的成员在派生类访问方式就不同了。具体方法如下。在派生类申明中，可以使用：using 基类::成员。

通过 using 申明来改变属性的规则如下：

- (1) 在基类中的private成员，不能在派生类中任何地方用using声明。
 - (2) 在基类中的protected成员和public成员，可以在派生类中任何地方用using声明。可以将该成员的访问属性在派生类中改为相应的public，protected和private 等。
- 实例 3-4 基类中的公有成员在私有继承后，在派生类中将其属性改为公有的方法。

```

#include <iostream>
using namespace std;

class CAnimal
{
public:
    CAnimal(float iWeight = 50)
    {m_fWeight = iWeight;}
    float GetWeight();
private:
    float m_fWeight;
};

float CAnimal::GetWeight()
{
    return m_fWeight;
}

class CPig:private CAnimal
{
public:
    using CAnimal::GetWeight;    //私有访问属性公有化
    float GetPorkWeight();
    CPig(float iPorkWeight = 30)
    {m_fPorkWeight = iPorkWeight;}
private:
    float m_fPorkWeight;
};

float CPig::GetPorkWeight()
{
    return m_fPorkWeight;
}

void main(void)
{
    CPig apig;
    cout<<apig.GetWeight()<<endl;
    cout<<apig.GetPorkWeight()<<endl;
}

```

3.4 派生类的构造函数和析构函数

前面介绍过，基类的构造函数无法继承。而且，用户在声明类时没有定义构造函数，系统会自动产生默认构造函数，构建对象时会自动调用这个默认构造函数。

如果用户需要定义派生类的构造函数，又该如何定义呢？派生类构造函数不仅要派生类新增的数据成员进行初始化，还要对基类的数据成员进行初始化。而且，派生类的继承方式不同，其构造函数的定义方法有所不同。下面按照不同的继承方式分别介绍派生类构造函数的定义方法。

3.4.1 单继承的派生类构造函数

任何派生类都必须继承基类成员（构造和析构函数除外），而且派生类的数据成员中不包含基类的对象（即子对象）。这种情况下的派生类构造函数定义的一般格式为：

派生类构造函数名（总参数表列）：基类构造函数名（参数表列）{派生类中新增数据成员初始化语句}。

对比分析发现：派生类构造函数对基类成员初始化，是采用构造函数初始化表方式。或者说，采用初始化表方式，不仅可以对构造函数的数据成员初始化，而且还可以调用基类构造函数来对基类数据成员的进行初始化。

实例 3-5 单继承的派生类构造函数应用举例。

```
#include <iostream>
#include<string>
using namespace std;

class Employee
{
public:
    Employee(int n,string nam,char s)    //基类构造函数
    {
        num=n;
        na =nam;
        sex=s;
    }
    ~Employee( ) { } //基类析构函数
protected :
    int num;
    string na ;
    char sex ;
};

class Intern: public Employee    //声明公用派生类
```

```

{
public :
    Intern(int n, string nam, char s, int a, string ad):Employee(n, nam, s) //派生类构造函数,
    通过初始化表调用基类构造函数
    {
        age=a;    //在函数体中只对派生类新增的数据成员初始化
        addr=ad;
    }
    void display( )
    {
        cout<<"num: "<<num<<endl;
        cout<<"name: "<<na <<endl;
        cout<<"sex: "<<sex<<endl;
        cout<<"age: "<<age<<endl;
        cout<<"address: "<<addr<<endl<<endl;
    }
    ~Intern( ) { } //派生类析构函数
private :
    int age;
    string addr;
};

int main( )
{
    Intern stud1(6688, "Zhang li", 'f', 18, "100 Tsinghua Road, Beijing");
    Intern stud2(6699, "Li gan", 'm', 20, "110 Chengfui Road, Beijing");
    stud1. display ( ); //显示对象stud1的数据
    stud2. display ( ); //显示对象stud2的数据
    return 0;
}

```

通过分析上述程序，可以总结出如下几点规则

(1) 构造函数的参数传递过程。派生类在构造对象时来完成实参到形参数的传递。在main()函数中，建立对象stud1时指定了5个实参。这5个实际参数按顺序传递给派生类构造函数Intern的形参。再由派生类构造函数将前面3个实参传递给基类构造函数相对应的形参。

(2) 派生类构造函数的申明与定义分离。虽然上面的程序实例中，派生类构造函数的申明和定义都在类内完成的。但也可以在类内申明，在类外面定义。具体格式如下。

类中申明构造函数形式如下：

```
Intern(int n, string nam, char s, int a, string ad);
```

请注意：在类中对派生类构造函数作声明时，不需要列出基类构造函数名及其参数表列。

类外定义派生类构造函数形式如下：

```
Intern::Intern(int n, string nam, char s, int a, string ad):Employee(n, nam, s)
{age=a;
addr=ad; }
```

(3) 基类构造函数的直接调用。调用基类构造函数时可直接使用常量或全局变量。例如，派生类构造函数首行可以写成以下形式：

```
Intern(string nam, char s, int a, string ad): Employee(6688, nam, s)
```

上述基类构造函数 3 个实参中，有 1 个参数是常量（6688），另外 2 个从派生类构造函数传递过来

请同学们考虑，是否可以将例 3.5 中派生类的基类构造函数的定义形式改写为 Intern(int n, string nam, char s, int a, string ad): Employee(n, nam, s), age(a), addr(ad) {}

(4) 派生类在构建对象时，构造函数的顺序是：①先调用基类构造函数；②再执行派生类构造函数本身（即派生类构造函数的函数体）。对上例来说，先初始化 num, name, sex，然后再初始化 age 和 addr。

(5) 派生类在析构对象时，先执行派生类析构函数 ~Intern()，再执行其基类析构函数 ~Employee()。

3.4.2 内嵌子对象的派生类的构造函数

类的数据成员中还可以包含类对象，即类对象中的内嵌对象，称为子对象(sub-object)。包含子对象的派生类的构造函数的初始化包括 3 部分：(1) 对基类数据成员初始化；(2) 对子对象数据成员初始化；(3) 对派生类新增数据成员初始化。

派生类构造函数的一般格式：

派生类构造函数名(总参数表列):基类构造函数名(参数表列),子对象名(参数表列){派生类中新增数据成员初始化语句}

其中：派生类构造函数的总参数表列中的参数，应当包括基类构造函数和子对象的参数表列中的参数。

执行顺序如下：

- ① 通过调用基类构造函数来对基类数据成员初始化；
- ② 通过调用子对象构造函数来对子对象数据成员初始化；
- ③ 再通过执行派生类构造函数本身来对派生类新增数据成员初始化。

下面介绍程序实例来说明包含子对象的派生类的构造函数的定义方法。

实例 3-6 包含子对象的派生类的构造函数应用举例。

为了简化程序以易于阅读，这里设基类 Employee 的数据成员只有两个，即 num 和 name。

```

#include <iostream>
#include <string>
using namespace std;

class Employee
{
public :
    Employee(int n, string nam ) //基类构造函数
    {
        num=n;
        na =nam;
    }
    void display( )
    {cout<<"num:"<<num<<endl<<"name:"<<na <<endl;}
protected :
    int num;
    string na ;
};

class Intern: public Employee //声明公用派生类
{
public:
    Intern(int n, string nam, int n1, string nam1,int a, string ad):
    Employee(n,nam),mentor(n1,nam1) //派生类构造函数
    {
        age=a;
        addr=ad;
    }
    void show( )
    {
        cout<<"This employee is:"<<endl;
        display();
        cout<<"age: " <<age<<endl;
        cout<<"address: " <<addr<<endl<<endl;
    }
    void show_mentor( ) //成员函数输出子对象
    {
        cout<<endl<<"Mentor is:"<<endl;
        mentor.display( ); //调用基类成员函数
    }
private :
    Employee mentor; //定义子对象
    int age;
    string addr;
};

```



```
};

int main( )
{
    Intern stud1(6688, "Wang-li", 6699, "Li-sun", 19, "115 Beijing Road, Shanghai");
    stud1.show( );
    stud1.show_mentor(); //输出子对象的数据
    return 0;
}
```

在上面的派生类构造函数中有6个形参，前2个作为基类构造函数的参数，第3和4个作为子对象构造函数的参数，第5、6个是用作派生类新增数据成员初始化的。这些参数的传递是按照相应的参数名称(而不是根据参数的顺序)来确立它们的传递关系的。因此，基类构造函数和子对象的次序可以是任意的(只是习惯上一般先写基类构造函数)。上面的派生类构造函数首部可以写成如下形式：

```
Intern(int n, string nam, int n1, string nam1, int a, string ad):
mentor(n1, nam1), Student(n, nam);
```

3.4.3 多级派生类的构造函数

一个类不仅可以派生出一个派生类，而且派生类还可以还做基类继续派生，形成派生类的多级结构。

多级派生类的构造函数一般形式：构造函数（参数总表）：直接基类构造函数。

下面通过实例来介绍多级派生类的构造函数的定义方法。

例 3-7 多级派生类的构造函数。

```
#include <iostream>
#include<string>
using namespace std;

class Employee
{
public :
    Employee(int n, string nam) //基类构造函数
    {
        num=n;
        name=nam;
    }
    void display( )
```

```

    {
        cout<<"num:"<<num<<endl;
        cout<<"name:"<<name<<endl;
    }
protected :
    int num;
    string name;
};

class Intern: public Employee //声明公用派生类
{
public :
    Intern(int n,string nam,int a):Employee(n,nam) //派生类构造函数
    {age=a; } //在此处只对派生类新增的数据成员初始化
    void show( )
    {
        display( );
        cout<<"age: "<<age<<endl;
    }
private :
    int age; //增加一个数据成员
};

class Female_Intern :public Intern //声明间接公用派生类
{
public:
    Female_Intern(int n, string nam,int a,int s):Intern(n,nam,a){score=s;}
    void show_all( ) //输出全部数据成员
    {
        show( );
        cout<<"score:"<<score<<endl;
    }
private:
    int score; //增加一个数据成员
};

int main( )
{
    Female_Intern stud(6688,"Li",19,99);
    stud.show_all( ); //输出全部数据
    return 0;
}

```

运行时的输出如下：

num:6688

name:Li

age:19

score:99

程序实例3-7中的二级派生关系为:Employee是Intern的直接基类,而Intern是Employee的直接派生类; Intern是Female_Intern的直接基类, Female_Intern是Intern直接派生类; Employee是Female_Intern的间接基类, Intern是Employee的间接派生类。

分析上述程序的执行过程可以知道: 在声明Female_Intern类对象时, 调用Female_Intern构造函数; 在执行Female_Intern构造函数时, 先调用Intern构造函数; 在执行Intern构造函数时, 先调用基类Employee构造函数。

因此, Female_Intern 类在构建对象时的初始化顺序如下:

- ① 先初始化基类的数据成员 num 和 name。
- ② 再初始化 Intern 的数据成员 age。
- ③ 最后再初始化 Female_Intern 的数据成员 score。

3.4.4 派生类构造函数的定义规则总结

除了上述规则之外, 在定义派生类构造函数时, 有以下几点规则需要注意:

(1) 当派生类新增的成员不需要初始化时, 派生类构造函数的函数体可以为空, 即构造函数是空函数。此派生类构造函数的主要功能只是将参数传递给基类和子对象的构造函数, 并调用基类构造函数和子对象构造函数。该用法常见于实际编程中。

(2) 如果在基类(子对象类型)中没有定义构造函数, 或者是定义了无参构造函数, 那么在定义派生类构造函数时可不写基类(对对象)构造函数。因为此时派生类构造函数没有向基类(子对象)构造函数传递参数的任务。此时系统会自动调用它们的默认构造函数。而且, 如果此时也不需对派生类自己的数据成员初始化, 则不必显式地定义派生类构造函数。在建立派生类对象时, 系统会自动调用系统提供的派生类的默认构造函数, 还会调用基类的默认构造函数和子对象类型默认构造函数。

(3) 相反, 如果在基类或子对象类型的声明中定义了带参数的构造函数, 那么就必须显式地定义派生类构造函数。

(4) 如果在基类中既定义无参的构造函数, 又定义了有参的构造函数, 则在定义派生类构造函数时, 既可以包含基类构造函数及其参数, 也可以不包含基类构造函数。派生类构造函数调用时, 根据构造函数的内容决定调用基类哪种构造函数。实际编程时, 根据需要来确定。

3.4.5 派生类的析构函数

派生类也不能继承基类的析构函数。因此，在派生类中，还需要定义析构函数。派生类的析构函数主要功能体现为：（1）根据需要对派生类中所增加的成员进行清理工作；（2）调用基类和子对象的析构函数，对基类和子对象的数据成员进行析构。

派生类的析构函数执行顺序：（1）执行派生类自己的析构函数，对派生类新增加的成员进行清理；（2）调用子对象的析构函数，对子对象进行清理；（3）调用基类的析构函数，对基类进行清理。

3.5 多重继承与虚基类

所谓多继承(multiple inheritance)是指 一个派生类有两个或多个基类，派生类从两个或多个基类中继承所需的属性。C++提供了多继承机制。多继承机制下的派生类一般申明格式如下。

```
class D:public A, private B, protected C //假设已声明了类 A、B 和 C
{类 D 新增加的成员}
```

D 是多重继承的派生类，它以公用继承方式继承 A 类，以私有继承方式继承 B 类，以保护继承方式继承 C 类。D 按不同的继承方式的规则继承 A,B,C 的属性，确定各基类的成员在派生类中的访问权限。图 3-5 是一种典型的多继承关系。

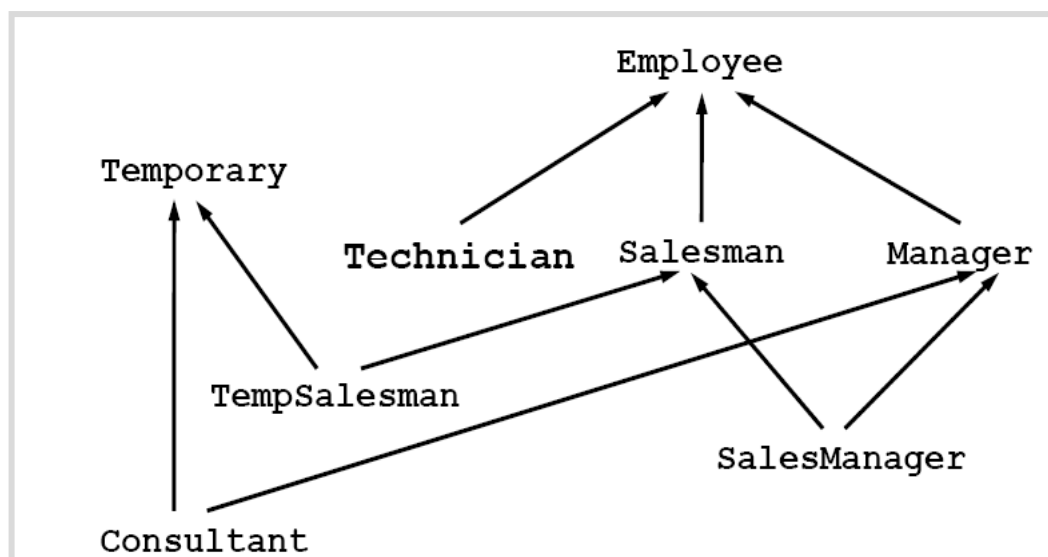


图3-5 一种典型的多继承关系

根据图 3-5 选取其中的一种关系来说明多重继承的派生类申明方法。TempSalesman 类是 Temporary 类和 Salesman 类的共同派生的。其申明方法可以用图 3-6 所示。

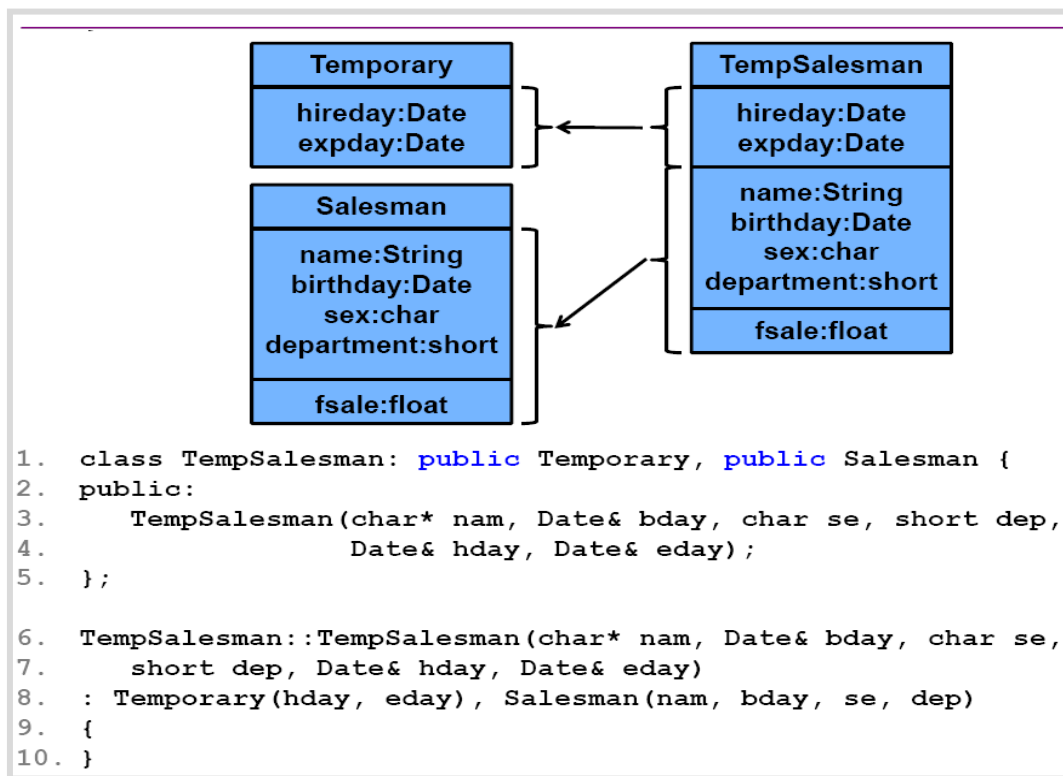


图 3-6 多重派生类的 TempSalesman 类申明方法

3.5.1 多重派生类构造函数的定义方法

从图 3-6 可以看出,多重继承派生类的构造函数形式与单继承时的构造函数形式基本相同,只是在初始表中包含多个基类构造函数。其申明一般格式如下所示。

派生类构造函数名(总参数表列):基类 1 构造函数(参数表列),基类 2 构造函数(参数表列),基类 3 构造函数(参数表列)

{派生类中新增数据成员初始化语句}

各基类的排列顺序任意。派生类构造函数的执行顺序同样为:先调用基类的构造函数,再执行派生类构造函数的函数体。调用基类构造函数的顺序是按照声明派生类时基类出现的顺序。

下面通过一个完整的程序来说明多重派生类的申明以及构造函数的基本方法。

实例 3-8 声明一个教师(Teacher)类和一个学生(Student)类,用多重继承的方式声明一个研究生(Graduate_Student)派生类。

```

#include<iostream>
#include<string>
using namespace std;

class teacher //基类1

```

```

{
public:
    teacher(string nam, int old, string adr)    //构造函数
    {
        name=nam;
        age=old;
        title=adr;
    }
    void display()
    {
        cout<<"name:"<<name<<endl;
        cout<<"age:"<<age<<endl;
        cout<<"title:"<<title<<endl;
    }
protected:
    string name;
    int age;
    string title;
};

class student    //基类2
{
public:
    student(char nam[], char s, float sco)    //基类2的构造函数
    {
        name1 = string(nam);
        sex=s;
        score=sco;
    }
    void display1()
    {
        cout<<"name:"<<name1<<endl;
        cout<<"sex:"<<sex<<endl;
        cout<<"score:"<<score<<endl;
    }
protected:
    string name1;
    char sex;
    float score;
};

class graduate_Student: public teacher, public student    //以公有的方式分别从teacher 和
student继承
{
public:
    graduate_Student (string nam, int old, char s, string adr, float sco, float w):

```

```

    teacher(nam, old, adr), student(nam, s, sco), pay (w)
{
}
void display2()
{
    cout<<"name:"<<name<<endl;
    cout<<"age:"<<age<<endl;
    cout<<"sex:"<<sex<<endl;
    cout<<"score:"<<score<<endl;
    cout<<"title:"<<title<<endl;
    cout<<" pay:"<< pay <<endl;
}
private:
    float pay;
};

```

分析上述程序实例，需要特别注意如下问题。

(1) 多重继承中的重复数据成员的问题。在两个基类中分别用name和name1来代表姓名，其实这是同一个人的名字，从Graduate_Student类的构造函数中可以看到总参数表中的参数nam分别传递给两个基类的构造函数，作为基类构造函数的实参。如何解决问题，请同学们思考。后面章节将进行深入介绍。

(2) 在设计派生类时要细致考虑其数据成员，尽量减少数据冗余。

3.5.2 多重继承中同名数据引起的二义性问题

多重继承可以反映现实生活中的情况，能够有效地处理一些较复杂的问题，使编写程序具有灵活性，但是多重继承也引起了一些值得注意的问题，它增加了程序的复杂度，使程序的编写和维护变得相对困难，容易出错。其中最常见的问题就是继承的成员同名而产生的二义性(ambiguous)问题。

假设类A和类B中都有成员函数display和数据成员a，类C是类A和类B的直接派生类。3个类的关系如图3-7所示。A, B, C 3个类的申明代码如下程序实例3-9所示。

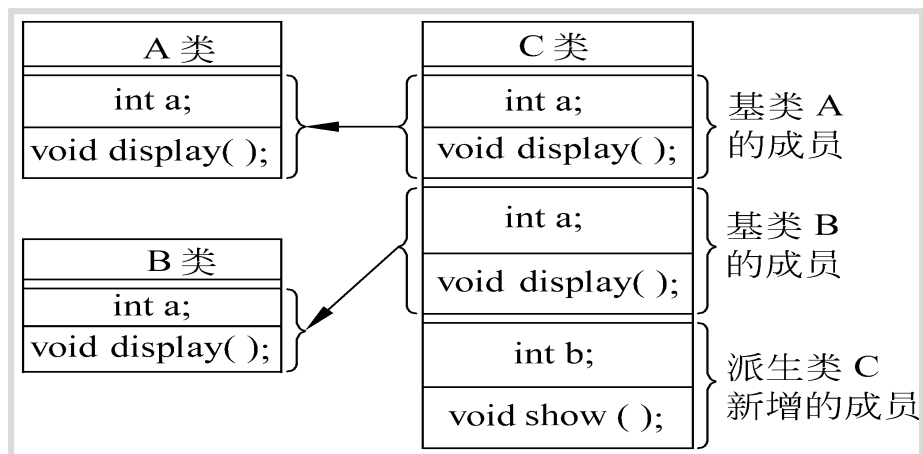


图3-7 多重继承中同名问题

实例 3-9 多重继承中的同名问题举例

```

class A
{
public:
    int a;
    void display( );
};

class B
{
public:
    int a;
    void display( );
};

class C :public A,public B
{
public :
    int b;
    void show();
};

int main()
{
    C c1;
    //c1.a=5;      //引用c1对象中的数据成员a。错误
    //c1.display(); //引用c1对象中的成员函数display()。错误
    return 0;
}
  
```

上述程序在编译时会出错。因为基类 A 和基类 B 都有数据成员 a 和成员函数 display，

编译系统无法判别要访问的是哪一基类的成员。这就是多重继承中的同名成员引起的二义性问题。

提问：假设构建 C 类对象 C1，则 C1 的同名成员的存储空间如何分配？为什么？

那么如何解决多重继承中的同名数据引起的二义性问题呢？解决二义性问题的根本措施是保证派生类成员的表示唯一性。具体方法有如下 3 种。

(1) 基类名的限定机制。可以采用基类名来限定类成员所在的类，实现成员名表示唯一性。

例如 `c1.A::a=53;` //引用 c1 对象中的基类 A 的数据成员 a

`c1.A::display();` //调用 c1 对象中的基类 A 的成员函数 display

(2) 派生类的同名覆盖机制。对于上述的程序实例 3-9，假设两个基类和派生类三者都有同名成员。例如将上面的 C 类声明改为：

```
class C :public A,public B
{
    int a;
    void display();
};
```

另外，如果在 main 函数中定义 C 类对象 c1，并调用数据成员 a 和成员函数 display。即：`c1.a=5; c1.display();` 此时程序能通过编译，也可正常运行。

因为，C++ 引入了同名覆盖机制，当派生类新增的成员与基类成员同名时，则基类的同名成员在派生类中被屏蔽，成为“不可见”的，或者说，派生类新增加的同名成员覆盖了基类中的同名成员。因此，`c1.a=5; c1.display();` 这 2 条语句访问的是派生类 C 中的新增成员，即派生类优先使用原则。

需要特别注意：不同的成员函数，只有在函数名和参数个数相同、类型相匹配的情况下才发生同名覆盖。如果只有函数名相同而参数不同，不会发生同名覆盖，而属于函数重载。

(3) 虚基类机制。解决多重继承中的同名成员引入的二义性问题，C++ 还提供了一种非常普遍使用的机制，即虚基类机制。有关虚基类的概念，将在下一节中具体介绍。

3.5.3 虚基类

在学习虚基类之前，先通过程序实例引入一种常见的多重继承中同名问题。

假设基类 A 派生了类 B 和类 C，然后，类 C 和 B 又共同派生类 D。这样，派生了类 B 和类 C 都包含有类 A 的成员。这样类 B 和类 C 派生类 D 时，就有 2 套同名成员。如图 3-10 所示。

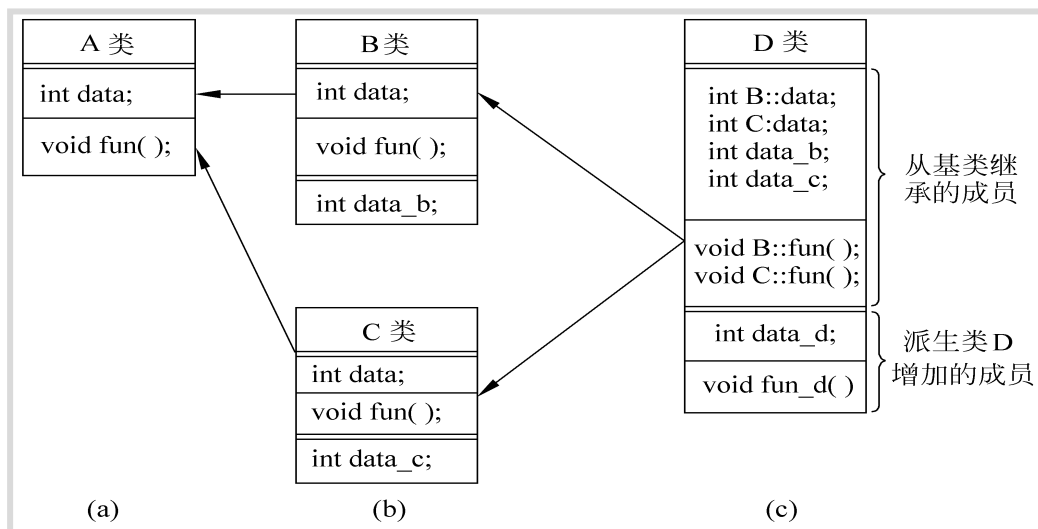


图3-7 多重继承的二义性问题

当然，解决派生类 D 中重名二义性问题，可以使用基类限定符和同名覆盖机制来实现。但这种情况下，更多使用的虚基类机制。

所谓虚基类机制是指 C++ 提供 virtual base class 方法，使得在继承相同间接基类时只保留一份成员，即同名成员在内存中只有 1 份拷贝。

声明虚基类的一般形式为：

class 派生类名: virtual 继承方式 基类名

例如，对于图 3.7 中的多重继承的二义性问题。如果采用如下的虚基类派生申明方法，就可以解决。

现在，将类 A 声明为虚基类，方法如下：

```
class A //声明基类 A
{...};

class B :virtual public A //声明类 B 是类 A 的公用派生类，A 是 B 的虚基类
{...};

class C :virtual public A //声明类 C 是类 A 的公用派生类，A 是 C 的虚基类
{...};

class D: public B, public C
{...};
```

按照上述虚基类的继承机制，派生类 D 就不会出现 2 套同名的成员。如图 3-11 所示。

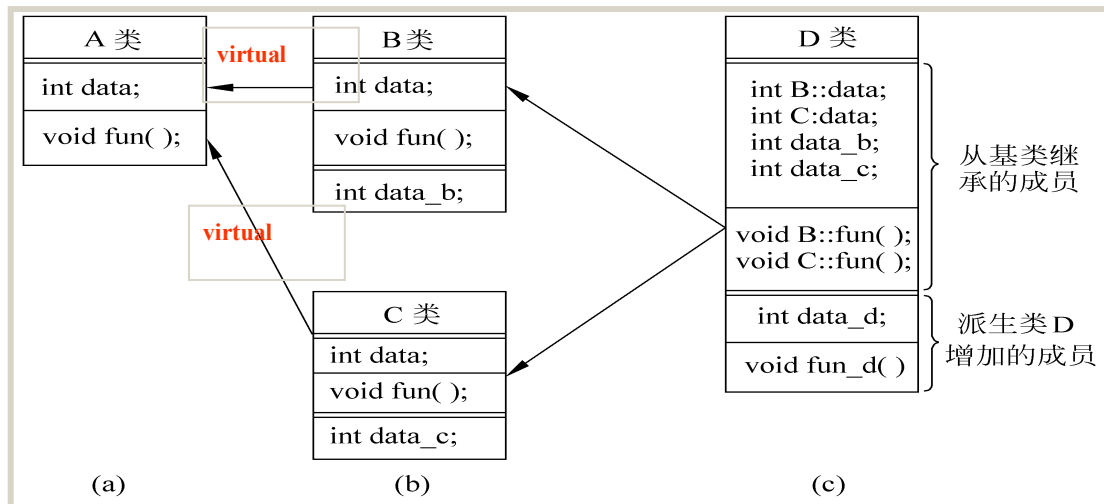


图 3-11 虚基类的继承机制

在理解虚基类时，需要关注如下几点。

(1) 虚基类不是在声明基类时声明的，而是在声明派生类时声明的。因此，对于一个基类来说，可以在生成这个派生类时作为虚基类，而在生成另个派生类时不作为虚基类。

(2) 为了保证虚基类在派生类中只有一次副本，应当在该基类的所有直接派生类中声明为虚基类。否则有可能会对基类的多次继承。例如，假设在派生类 B 和 C 中将类 A 声明为虚基类，而在派生类 D 中没有将类 A 声明为虚基类。在派生类 E 中，虽然从类 B 和 C 派生路径只保留一份基类成员副本，但从类 D 派生路径还保留一份基类成员副本。

(3) 虚基类的初始化。如果在虚基类中定义了有参构造函数，则在其所有派生类(包括直接或间接派生类)中，使用构造函数的初始化表来初始化虚基类。而且构造函数的使用方法与以往不同，即要求在最后的派生类中不仅要对其直接基类进行初始化，还要对虚基类初始化。例如，上述实例中类 D 的构造函数定义形式为：D(int n):A(n),B(n),C(n){ }。注意：在执行时，C++编译系统只执行最后的派生类对虚基类构造函数的调用，而忽略其他派生类(如类 B 和类 C)对虚基类构造函数的调用，以此来确保虚基类只初始化 1 次。

下面通过一个完整的程序实例 3-10 来说明虚基类的应用方法。

```
#include <iostream>
#include <string>
using namespace std;

class Person //声明公共基类Person
{
public:
    Person(string nam,char s,int a) //构造函数
    {name=nam;sex=s;age=a;}
protected :
    string name;
    char sex;
    int age;
```

```

};

class Teacher:virtual public Person    //声明Person为公用继承虚基类
{
public :
    Teacher(string nam,char s,int a, string t):Person(nam,s,a)    //构造函数
    {title=t;}
protected :
    string title;
};

class Student:virtual public Person //声明Person为公用继承虚基类
{
public:
Student(string nam,char s,int a,float sco) :Person(nam,s,a),score(sco){ }    //构造函数
protected :
    float score;
};

class Graduate:public Teacher, public Student    //Teacher和Student为直接基类
{
public:
    Graduate(string nam,char s,int a, string t,float sco,float w):Person(nam,s,a),
Teacher(nam,s,a,t), Student(nam,s,a,sco), pay (w){}    //构造函数
    void display( )
    {
        cout<<"name:"<<name<<endl;
        cout<<"age:"<<age<<endl;
        cout<<"sex:"<<sex<<endl;
        cout<<"score:"<<score<<endl;
        cout<<"title:"<<title<<endl;
        cout<<"wages:"<< pay <<endl;
    }
private:
    float pay;
};

int main( )
{
    Graduate grad1("Wang-li",'f',24,"assistant",89.5,1234.5);
    grad1. display ( );
    return 0;
}

```

运行结果为

```
name: Wang-li
age:24
sex:f
score:89.5
title:assistant
pay:1234.5
```

从上述程序可以看到：通过 `grad1.display()` 的函数体，可以唯一地访问 `Person` 类中的 `name`、`sex` 和 `age` 3 个数据成员。虚基类机制能很好地解决多重继承中同名成员的二义性问题。

为了进一步说明虚基类在继承和派生中的特点，即基类成员的唯一性问题。下面列举了实例 3-11。在下列程序中，类 `B` 和 `C` 分别以虚基类的继承方式从基类 `A` 派生而成，然后，又以 `B` 和 `C` 为基类，派生了类 `D`。在主函数中，类 `D` 构建了一个对象 `objD`。

实例 3-11 虚基类的使用

```
#include <iostream>
using namespace std;

class A
{
public:
    A(void)
    {a=10;}
    void Func(void)
    {cout<<"Func of A"<<endl;}
protected :
    int a;
};

class B : virtual public A
{
public:
    B( void )
    {
        a += 10;
        cout <<"Ba = "<< a <<' \n' ;
    }
};

class C : virtual public A
{
public:
    C( void )
    {
```

```

        a += 10;
        cout <<"Ba = "<< a <<'\\n';
    }
};

class D : B, C
{
public:
    D( void )
    {
        cout <<"a = "<< a << endl;
    }
};

void main()
{
    D objD;
}

```

运行结果

Ba = 20

Ba = 30

a = 30

上述程序在构建对象 objD 时，自动调用类 D 的构造函数。然而，类 D 的构造函数执行顺序是先执行类 A 的构造函数，然后是类 B 和类 C 的构造函数，最后执行类 D 的构造函数函数体。在分析上述实例时，注意 2 点。

- (1) 派生类的构造函数执行顺序；
- (2) 派生类的构造函数申明方法，在基类为无参构造函数时，可以不必在派生类的构造函数中显式调用，但会隐含自动调用。
- (3) 虚基类在派生类 D 对象只有一份成员拷贝，因此，类 B 和 C 的构造函数的修改都是针对一个变量来操作的。

建议：不提倡在程序中使用多重继承，只有在不易出现二义性时或实在必要时才使用多重继承。因此许多多重继承问题可以采用单一继承解决。因此，现在有些面向对象的程序设计语言(如 Java)不支持多重继承。

另外，我们分析派生类构造函数执行顺序时，需要重点关注虚基类和非虚基类构造函数的自动调用顺序。基本规则如下：

- (1) 虚基类的构造函数在非虚基类之前调用
- (2) 同一层包含多个虚基类，则按申明顺序调用

(3) 若虚基类由非虚基类派生而来，则要先调用更高级别的基类构造函数，再遵循(1)和(2)的顺序。

下面通过程序实例 3-12 来说明虚基类的构造函数自动调用顺序。

实例 3-12 虚基类和非虚基类的构造函数自动调用顺序

```
#include <iostream>
using namespace std;

class Base1
{
public:
    Base1( void )
    {cout <<"class Base1"<<endl;}
};

class Base2
{
public:
    Base2( void )
    {cout <<"class Base2"<<endl;}
};

class Level1 : public Base2, virtual public Base1
{
public:
    Level1 ( void )
    {cout <<"class Level1"<<endl;}
};

class Level2 : public Base2, virtual public Base1
{
public:
    Level2 ( void )
    {cout <<"class Level2"<<endl;}
};

class Leaf : public Level1, virtual public Level2
{
public:
    Leaf ( void )
    {cout << "class Leaf" <<endl;}
};
```

```
void main(void)
{
    Leaf obj;
}
```

程序运行结果

```
Class Base1
Class Base2
Class Level12
Class Base2
Class level11
Class Leaf
```

在上述程序中，在主函数 main（）构建对象 obj 时，自动调用 Leaf 类的构造函数，然而，Leaf 构造函数会优先自动执行直接虚基类 Level12 的构造函数。在执行 Level12 的构造函数时，则优先自动执行 Level12 的直接虚基类 Base1 的构造函数，然后执行非虚基类 Base2 的构造函数，接着执行 Level12 的构造函数；然后再自动执行 Leaf 类的直接非虚基类 Level11 的构造函数；在执行 Level11 的构造函数时，因为直接虚基类 Base1 的构造函数已经执行了，按照虚基类成员只有 1 份拷贝的特点，就不再执行了。直接执行非虚基类 Base2 的构造函数，再接着执行 Level11 的构造函数，以及 Leaf 类的构造函数。

3.6 继承与组合

前面主要介绍类的继承与派生。在 C++ 代码重用机制中，还有一种方法是类的组合。类的组合 (composition) 是指在一个类中以另一个类的对象作为数据成员。这就是所谓类中的子对象，类似结构体的嵌套申明。

在自然界中，有些类与类之间没有继承与派生关系，也就是说，这些类不是属于同一系族，属于不同范畴的对象。例如，从一个家庭系统来说，父亲与孩子这 2 种类有继承与派生关系，但“父亲类”和“孩子类”组成“家庭类”时，“孩子类”和““父亲类”都与“家庭类”都没有继承与派生关系。它们只存在组合关系。因此，类的组合和继承一样，都是有效地利用已有类的资源，实现代码的重用。但二者有着本质区别：继承是纵向的（属于同范畴），组合是横向的（属于不同范畴）。

下面通过程序实例来说明包含子对象的类申明和引用方法。

实例 3-13. 假设已经声明了一个学生类 student，在此基础上，通过 student 类的继承，申明了一个派生类 student1。在派生类 student1 中，新增一个数据成员 monitor。该成员是类 student 的一个对象，称为 student1 的一个子对象的数据成员。因此， student1 类

既是基类 student 的派生，也是类 student 的组合。

```
#include <iostream>
#include <string>
using namespace std;

class Employee//声明基类
{
public:
    Employee(int n, string nam) //基类构造函数
    {
        num = n;
        name = nam;
    }
    void display()
    {
        cout<<"num:"<<num<<endl;
        cout<<"name:"<<name<<endl;
    }
protected:
    int num;
    string name;
};

class Intern : public Employee//声明公用派生类Intern
{
public:
    Intern(int n, string nam, int n1, string nam1, int a, string ad)
        :Employee(n, nam), mentor(n1, nam1) //派生类构造函数
    {age = a; addr = ad;}
    void show()
    {
        cout<<"This employee is:"<<endl;
        display();
        cout<<"age:"<<age<<endl;
        cout<<"address:"<<addr<<endl<<endl;
    }
    void show_mentor()
    {
        cout<<endl<<"Mentor is:"<<endl;
        mentor.display(); //调用基类成员函数
    }
private:
    Employee mentor; //定义子对象
    int age;
```

```

        string addr;
    };

    int main()
    {
        Intern stud1(7788, "Wang-li", 7878, "Li-sun", 19,
            "115 Beijing Road, Shanghai");
        stud1.show();
        stud1.show_mentor();
        return 0;
    }

```

在上述程序中，需要特别关注的是在组合类下的构造函数的定义方法。在这前面的章节进行了介绍。在此不在赘述。

下面通过程序实例 3-14 来进行说明组合类的使用方法。同时注意其中类中静态数据成员的使用。

实例 3-14 组合类的应用

//类声明文件名: companyStaff.h

```

// 编译预处理语句: 如果未定义__COMPANYSTAFF_H__ 则定义它,
// 直到遇到#endif结束。
#ifndef __ COMPANYSTAFF_H__
#define __ COMPANYSTAFF_H__

#include<iostream>    //包含头文件, 使用iostream库用于输入输出
#include<string>      //包含头文件, 使用字符串处理函数
using namespace std; //使用std名字空间

const int LEN = 50; //定义一个常量, 表示字符数组长度

// CEducation类的声明

class CEducation    // 教育背景
{
public:
    CEducation() {} ; // 构造函数

    CEducation( char cSchool[], char cDegree )// 重载构造函数, 为数据成员赋初值
    {
        strcpy( m_cSchool, cSchool );
        m_cDegree = cDegree;
    }
}

```

```

    void GetEdu( void );           // 输入教育背景的有关数据

    void PutEdu( void );           // 输出教育背景的有关信息

private:
    char m_cSchool[LEN];           // 毕业学校
    char m_cDegree;                 // 最高学历：专科H、本科B、硕士M、博研D
};

// 组合类的声明

class CCompanyStaff               // 员工类
{
public:
    CCompanyStaff( void ){}        ;    // 构造函数

    // 重载构造函数，使用初始化列表对内嵌对象进行初始化
    CCompanyStaff( char s[], char d, char n[], float r, float b ): Edu(s,d)
    {
        m_iStaffNum = ++s_iCount;
        strcpy( m_cName, n );
        m_fRateOfAttend = r;
        m_fBasicSal = b;
    }

    ~CCompanyStaff( void ){ } ;    // 析构函数

    void CalculateSal( void ){ } ; // 计算实发工资

    void OutPut( void );           // 输出员工基本信息

    void InPut( void );            // 输入员工基本信息

protected:
    CEducation Edu;                // 内嵌对象（教育背景）
    int m_iStaffNum;                // 工作编号
    char m_cName[LEN];              // 姓名
    float m_fRateOfAttend;          // 出勤率
    float m_fBasicSal ;             // 基本工资
    static int s_iCount;            // 静态累加器
};

#endif //结束编译预处理

```

//主文件: main.cpp

#include "companyStaff.h"// 把类声明文件包含进来

// 测试程序

void main(void)

{

 char flag; // 设置判断是否继续录入的标志

 // 创建一个员工对象, 通过构造函数初始化数据

 CCompanyStaff Staff("tsinghua University", 'M', "SunLi", 1.0, 2000);

 Staff.OutPut();

 cout << "是否继续录入信息? (Y/N) ";

 cin >> flag;

 // 用toupper函数将用户输入的字符规范为大写字符来与'Y' 进行比较

 if (toupper(flag) == 'Y')

 {

 CCompanyStaff staffs[100]; // 创建一个能存放个员工对象的数组

 for (int i = 0; i < 100;) // 循环录入和输出数组中各员工对象的数据

 {

 staffs[i].InPut(); // 通过下标为i的员工对象调用接口InPut

 staffs[i].OutPut(); // 通过下标为i的员工对象调用接口OutPut

 cout << "是否继续录入信息? (Y/N) ";

 cin >> flag;

 if (toupper(flag) == 'N') // 循环录入的条件

 {

 break;

 }

 }

 }

}

// 类实现文件: companyStaff.cpp

#include "companyStaff.h" // 包含类声明文件

int CCompanyStaff::s_iCount = 1000; // 初始化静态数据成员s_iCount

// 实现类CEducation的成员函数GetEdu, 输入信息

```

void CEducation::GetEdu( void )
{
    cout << endl << " 毕业学校: ";
    cin >> m_cSchool;
    cout << endl << " 最高学历: (专科H、本科B、硕士M、博士D) ";
    cin >> m_cDegree;

    // 用toupper函数将用户输入的学历规范为大写字符
    m_cDegree = toupper( m_cDegree );
}

// 实现类CEducation的成员函数PutEdu, 输出信息
void CEducation::PutEdu( void )
{
    cout << endl << " 毕业学校: "<< m_cSchool << endl;
    cout << endl << " 最高学历: ";
    switch ( m_cDegree )
    {
        case 'H': cout << "专科"<< endl;
                  break;
        case 'B': cout << "本科"<< endl;
                  break;
        case 'M': cout << "硕士"<< endl;
                  break;
        case 'D': cout << "博士"<< endl;
                  break;
        default:  cout << "空"<< endl; // 输入的学历在选项之外则为"空"
                  break;
    }
}

// 实现类CCompanyStaff的成员函数InPut, 输入信息
void CCompanyStaff:: InPut( void )
{
    m_iStaffNum = ++s_iCount; // 静态累加器自加后赋给工作编号

    cout << endl << "请输入编号为"<< m_iStaffNum << "号员工的姓名: ";
    cin >> m_cName; // 输入员工姓名

    Edu.GetEdu(); // 通过内嵌对象调用类CEducation的接口GetEdu, 输入教育背景相关数据

    cout << endl << " 基本工资: ";
    cin >> m_fBasicSal; // 输入基本工资
}

```

```

cout << endl << " 出勤率: ";
cin >> m_fRateOfAttend; // 输入出勤率

// 规范用户输入的出勤率的值
if ( m_fRateOfAttend > 1.0 )
{
    m_fRateOfAttend = 1.0;
}
else
    if ( m_fRateOfAttend < 0 )
    {
        m_fRateOfAttend = 0;
    }
}

// 实现类CCompanyStaff的成员函数OutPut, 输出信息
void CCompanyStaff:: OutPut( void )
{
    cout << endl << "显示员工" << m_cName << "的基本数据: ";
    cout << endl << " 工作编号: " << m_iStaffNum;
    Edu.PutEdu(); // 通过内嵌对象调用类CEducation的接口PutEdu
    cout << endl << " 基本工资: " << m_fBasicSal;
    cout << endl << " 出勤率: " << m_fRateOfAttend * 100 << "%";
}

```

3.6 综合程序应用—公司人事管理系统

在全国人事改革浪潮的冲击下,某公司也计划进行人事改革,对员工设岗。初步确定设立3类岗位:经理(manager)1名,技术(technician)岗10名,其余为销售岗位(salesman)。岗位不同,待遇不同。经理拿固定月薪12000元,技术人员按每小时260元拿月薪,销售岗按当月的销售额提成5%。另外,增加一个销售经理(Salemanager)岗位,需从经理和销售人员中选拔。因此,该岗位具有经理和销售的共有特征。但月薪不同:既有固定月薪8000元,还从当月所管辖人员销售总额提成4%。要求如下:

- (1) 在前面一章的综合程序应用的基础上,对“公司人事管理系统”进行功能扩展和修改。采用类继承方式实现上述功能。
- (2) 要求采用多重继承和虚基类方法。

```

/*****

```

Filename: employee.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: employee类的声明

```

/*****/
#ifndef EMPLOYEE_H
#define EMPLOYEE_H

//employee类，对公司的雇员信息进行封装
class employee
{
private:
    //三个私有数据成员，分别代表雇员的编号(individualEmpNo)、等级(grade)与月薪(accumPay)
    int individualEmpNo;
    int grade;
    int accumPay;

    //这个静态私有成员用于在新加入雇员时使编号自动增加
    static int currentEmpNo;
public:
    //无参构造函数与有参构造函数
    employee();
    employee(int inputGrade, int inputPay);

    //析构函数
    ~employee();

    //访问数据成员的接口函数，分别是
    //设置等级: setGrade(int)
    //设置月薪: setAccumPay(int)
    //获取编号: int getEmpNo()
    //获取等级: int getGrade()
    //获取月薪: int getAccumPay()
    //注: 雇员编号是构造时自动生成的，为避免编号冲突，操作者无法设置编号
    //由于派生类的工资基于另外的规则进行设置，故不建议直接使用setAccumPay接口
    void setGrade(int inputGrade) {grade = inputGrade;};
    void setAccumPay(int inputPay) {accumPay = inputPay;};
    int getEmpNo() {return individualEmpNo;};
    int getGrade() {return grade;};
    int getAccumPay() {return accumPay;};

    //该函数的功能是打印雇员的基本信息

```

```

    virtual void printInfo();

    //流输入运算符重载，从流向对象输入基本信息
    friend std::istream& operator >>(std::istream&, employee&);

    //流输出运算符重载，向流输出对象的基本信息
    friend std::ostream& operator <<(std::ostream&, employee&);
};

#endif

/*****
Filename: linkedlist.h
Copyright: Tsinghua University
Author: Huaizhou Tao
Date: 2014-10-02
Description: linkedlist类的声明
*****/

#ifndef LINKEDLIST_H
#define LINKEDLIST_H

//基于模板的链表结点定义
template <class T>
struct node
{
    T data;
    node<T> *next;
};

//基于模板的链表类声明
template <class T>
class linkedlist
{
private:
    node<T> *head;        //头结点
    node<T> *current;     //当前结点
    //内联函数，用于深拷贝
    inline void deepCopy(const linkedlist<T> &original);
public:
    //构造函数，拷贝构造函数，析构函数
    linkedlist();
    linkedlist(const linkedlist<T> &aplist);
    ~linkedlist();

```



```

    void insert(const T &element);           //在头部之前插入元素
    void insert_end(const T &element);       //在尾部插入
    bool getFirst(T &listElement);          //获得链表头的数据
    inline bool getNext(T &listElement);     //获得当前结点的下一个数据
    bool find(const T &element);            //查找一个数据
    bool retrieve(T &element);              //检索一个数据
    bool replace(const T &newElement);      //替换一个数据
    bool remove(T &element);                //移除一个数据
    bool isEmpty() const;
    void makeEmpty();
    //重载"="运算符
    linkedlist<T>& operator =(const linkedlist<T> &rlist);
};

#endif

/*****
Filename:manager.h

Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: manager类的声明

*****/
#ifndef MANAGER_H
#define MANAGER_H

#define FIXED_PAY_MANAGER 12000

#include "employee.h"

//manager类，继承employee类，代表经理岗位
class manager: virtual public employee
{
private:
    //经理子类不需要更多的数据成员
public:
    //无参构造函数与有参构造函数
    manager();
    manager(int inputGrade);

    //析构函数

```

```

~manager();

//该函数的功能是打印雇员的基本信息
void printInfo();

//流输入运算符重载，从流向对象输入基本信息
friend std::istream& operator >>(std::istream&, manager&);

//流输出运算符重载，向流输出对象的基本信息
friend std::ostream& operator <<(std::ostream&, manager&);
};

#endif

/*****
Filename:salemanager
Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: salemanager类的声明

*****/

#ifndef SALEMANAGER_H
#define SALEMANAGER_H

#define FIXED_PAY_SALEMANAGER 8000
#define RATE_SALEMANAGER 0.04

#include "manager.h"
#include "salesman.h"
#include "linkedlist.cpp"

//salemanager类，继承manager类与salesman类，代表销售经理
class salemanager: public manager, public salesman
{
private:
    //使用链表保存经理手下管理的销售员的编号
    linkedlist<int> salesmanNoList;
public:
    //无参构造函数与有参构造函数
    salemanager();
    salemanager(int inputGrade, int inputSales, linkedlist<int> &inputNoList);

    //析构函数

```

```

~salemanager();

//成员函数：用于改变经理的销售总额
void changeSales(int inputSales);

//成员函数：用于添加或删除经理的下属销售员
bool addSalesman(int empNo);
bool deleteSalesman(int empNo);

//该函数的功能是打印雇员的基本信息
void printInfo();

//流输入运算符重载，从流向对象输入基本信息
friend std::istream& operator >>(std::istream&, salemanager&);

//流输出运算符重载，向流输出对象的基本信息
friend std::ostream& operator <<(std::ostream&, salemanager&);
};

#endif

/*****
Filename: salesman.h

Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: salesman类的声明

*****/
#ifndef SALESMAN_H
#define SALESMAN_H

#define RATE_SALESMAN 0.05

#include "employee.h"

class salemanager;

//salesman类，继承employee类，代表销售岗位
class salesman: virtual public employee
{
private:
    //新增的私有数据成员表示销售员的月销售额与提成率

```

```

    int sales;
    double commissionRate;

    //该成员表示管理该销售员的销售经理
    salemanager *boss;
public:
    //无参构造函数与有参构造函数
    salesman();
    salesman(int inputGrade, int inputSales, salemanager *inputBoss);

    //析构函数
    ~salesman();

    //访问数据成员的接口函数，分别是
    //设置提成比率: setCommissionRate(double)
    //设置销售额:   setSales(int)
    //设置销售经理: setBoss(salemanager*)
    //获取提成比率: double getCommissionRate()
    //获取销售额:   int getSales()
    //获取销售经理: salemanager* getBoss()
    void setCommissionRate(double inputRate) {commissionRate =
inputRate;setAccumPay((int) (sales * commissionRate));};
    void setSales(int inputSales);
    void setBoss(salemanager *inputBoss);
    double getCommissionRate() {return commissionRate;};
    int getSales() {return sales;};
    salemanager* getBoss() {return boss;};

    //该函数的功能是打印雇员的基本信息
    void printInfo();

    //流输入运算符重载，从流向对象输入基本信息
    friend std::istream& operator >>(std::istream&, salesman&);

    //流输出运算符重载，向流输出对象的基本信息
    friend std::ostream& operator <<(std::ostream&, salesman&);
};

#endif

/*****
Filename: technician.h

Copyright: Tsinghua University

```

Author: Huaizhou Tao

Date:2014-10-02

Description: technician类的声明

```

/*****/
#ifndef TECHNICIAN_H
#define TECHNICIAN_H

#define WAGE_TECHNICIAN 260

#include "employee.h"

//technician类，继承employee类，代表技术岗位
class technician: public employee
{
private:
    //新增的私有数据成员表示技术岗雇员的月工作时间与时薪
    int workHour;
    int wage;
public:
    //无参构造函数与有参构造函数
    technician();
    technician(int inputGrade, int inputWorkHour);

    //析构函数
    ~technician();

    //访问数据成员的接口函数，分别是
    //设置基本时薪: setWage(int)
    //设置工作时间: setWorkHour(int)
    //获取基本时薪: int getWage()
    //获取工作时间: int getWorkHour()
    void setWage(int inputWage) {wage = inputWage;setAccumPay(workHour * wage);};
    void setWorkHour(int inputWorkHour) {workHour = inputWorkHour;setAccumPay(workHour *
wage);};
    int getWage() {return wage;};
    int getWorkHour() {return workHour;};

    //该函数的功能是打印雇员的基本信息
    void printInfo();

    //流输入运算符重载，从流向对象输入基本信息
    friend std::istream& operator >>(std::istream&, technician&);
};

```

```

        //流输出运算符重载，向流输出对象的基本信息
        friend std::ostream& operator <<(std::ostream&, technician&);
    };

#endif

/*****
Filename:main.cpp

Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: 程序主函数文件

*****/
#include <iostream>
#include <fstream>
#include "employee.h"
#include "manager.h"
#include "technician.h"
#include "salesman.h"
#include "salemanager.h"
using namespace std;

#define NUM_SALEMNG 2
#define NUM_TECHNICIAN 10
#define NUM_SALESMAN 10

//Summary: 主函数
//
//Parameters:
//      None
//Return: int 一般为
int main()
{
    //公司现有人员构成：一位经理，两位销售经理，位技术员，位销售员
    manager mng;
    salemanager saleMng[NUM_SALEMNG];
    technician tech[NUM_TECHNICIAN];
    salesman slm[NUM_SALESMAN];

    //使用文件进行个人信息的输入、输出操作
    ifstream inputFile("emp_info.txt", ios::in);
    ofstream outputFile("emp_output.txt", ios::out);

```

```

int count;
//重载流输入操作符完成个人信息输入
inputFile>>mng;
inputFile>>saleMng[0];
inputFile>>saleMng[1];
for(count = 0; count < NUM_TECHNICIAN; count++)
    inputFile>>tech[count];
for(count = 0; count < NUM_SALESMAN; count++)
    inputFile>>slm[count];

//指定第一位销售经理管辖前五位销售员，第二位经理管辖其他
for(count = 0; count < NUM_SALESMAN / 2; count++)
    slm[count].setBoss(&saleMng[0]);
for(count = NUM_SALESMAN / 2; count < NUM_SALESMAN; count++)
    slm[count].setBoss(&saleMng[1]);

cout<<"将第一位销售员划归第二位经理管辖，最后一位销售员划归第一位经理管辖。"<<endl;
slm[0].setBoss(&saleMng[1]);
slm[NUM_SALESMAN - 1].setBoss(&saleMng[0]);
cout<<"改变第三位销售员的销售额"<<endl;
slm[2].setSales(520000);

cout<<"将所有雇员信息输出到文件。"<<endl;
outputFile<<mng;
outputFile<<saleMng[0];
outputFile<<saleMng[1];
for(count = 0; count < NUM_TECHNICIAN; count++)
    outputFile<<tech[count];
for(count = 0; count < NUM_SALESMAN; count++)
    outputFile<<slm[count];

//关闭文件
inputFile.close();
outputFile.close();

return 0;
}

```

```

/*****

```

Filename: employee.cpp

Copyright: Tsinghua University

Author: Huaizhou Tao

Date:2014-10-02

Description: employee类的定义

```

/*****
#include <iostream>
#include "employee.h"
using namespace std;

//设置静态变量的初始值
int employee::currentEmpNo = 2014001;

//Summary: 输出雇员的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 该函数首先输出一条提示信息，然后按照
//      编号、等级与月薪的顺序输出雇员的基本信息
void employee::printInfo()
{
    cout<<"输出个人信息: "<<endl;
    cout<<"编号: "<<individualEmpNo<<endl;
    cout<<"等级: "<<grade<<endl;
    cout<<"月薪: "<<accumPay<<endl<<endl;
}

//Summary: 无参构造函数
//
//Parameters:
//      None
//Return: None
//Detail: 无参构造函数首先完成新雇员的自动编号功能
//      再将雇员的等级设为，月薪设为
employee::employee()
{
    individualEmpNo = currentEmpNo;
    currentEmpNo++;
    grade = 1;
    accumPay = 3000;
}

//Summary: 无参构造函数
//

```



```

//Parameters:
//      inputGrade: 输入的雇员等级
//      inputPay:   输入的雇员月薪
//Return: None
//Detail: 有参构造函数首先完成新雇员的自动编号功能
//      之后根据输入的参数设置雇员等级与月薪
//注:    未添加合法性判断, 请勿输入非正数
employee::employee(int inputGrade, int inputPay)
{
    individualEmpNo = currentEmpNo;
    currentEmpNo++;
    grade = inputGrade;
    accumPay = inputPay;
}

//Summary: 析构函数
//
//Parameters:
//      None
//Return: None
//Detail: 释放对象, 并输出提示信息
employee::~~employee()
{
    cout<<"欢迎使用, 再见!"<<endl;
}

//Summary: 流输入操作符重载函数
//
//Parameters:
//      input: 输入流
//      e:     目标雇员对象
//Return: istream& 返回输入流本身
//Detail: 重载函数首先给出输入提示, 之后从流中输入目标对象的等级与月薪
//注:    未添加合法性判断, 请勿输入非正数
istream& operator >> (istream& input, employee& e)
{
    cout<<"输入编号为"<<e.individualEmpNo<<" 的雇员的等级、月薪: "<<endl;
    input>>e.grade;
    input>>e.accumPay;
    return input;
}

//Summary: 流输出操作符重载函数
//

```

```

//Parameters:
//      output: 输出流
//      e:      目标雇员对象
//Return: ostream& 返回输出流本身
ostream& operator << (ostream &output, employee &e)
{
    output<<"输出个人信息: "<<endl;
    output<<"编号: "<<e.individualEmpNo<<endl;
    output<<"等级: "<<e.grade<<endl;
    output<<"月薪: "<<e.accumPay<<endl<<endl;
    return output;
}

/*****
Filename:linkedlist.cpp

Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: linkedlist类的定义

*****/

#include "linkedlist.h"

//Summary: 无参构造函数
//
//Parameters:
//      None
//Return: None
template<class T>
linkedlist<T>::linkedlist()
{
    head = current = NULL;
}

//Summary: 拷贝构造函数
//
//Parameters:
//      aplist
//Return: None
template<class T>
linkedlist<T>::linkedlist(const linkedlist<T> &aplist)
{
    deepCopy(aplist);
}

```

```

}

//Summary: 析构函数
//
//Parameters:
//      None
//Return: None
template<class T>
linkedList<T>::~~linkedList()
{
    makeEmpty();
}

//Summary: 插入函数
//
//Parameters:
//      element 待插入元素
//Return: None
//Detail: 该函数在头部之前插入，插入后没有当前位置
template<class T>
void linkedList<T>::insert(const T &element)
{
    current = NULL;
    node<T> *newNode = new node<T>;
    newNode->data = element;
    newNode->next = head;
    head = newNode;
}

//Summary: 插入函数
//
//Parameters:
//      element 待插入元素
//Return: None
//Detail: 该函数在尾部插入新元素
template<class T>
void linkedList<T>::insert_end(const T &element)
{
    current = NULL;
    node<T> *newNode = new node<T>;
    node<T> *tail = head;
    newNode->data = element;
    newNode->next = NULL;
    if(tail == NULL)

```

```

    {
        head = newNode;
    }
    else
    {
        while(tail->next != NULL)
            tail = tail->next;
        tail->next = newNode;
    }
}

//Summary: 获得链表头的函数
//
//Parameters:
//    listElement 返回结果
//Return: bool
//Detail: 在链表为空时返回false
template<class T>
bool linkedlist<T>::getFirst(T &listElement)
{
    if(head == NULL)
        return false;
    current = head;
    listElement = head->data;
    return true;
}

//Summary: 获得下一个数据
//
//Parameters:
//    listElement 返回结果
//Return: bool
//Detail: current在执行后将指向下一个元素
template<class T>
bool linkedlist<T>::getNext(T &listElement)
{
    if(current == NULL)
        return false;
    if(current->next == NULL)
    {
        current = NULL;
        return false;
    }
    listElement = current->next->data;

```

```

        current = current->next;
        return true;
    }

//Summary: 查找一个数据
//
//Parameters:
//    element 目标数据
//Return: bool
//Detail:
template<class T>
bool linkedlist<T>::find(const T &element)
{
    T item;
    if(!getFirst(item))
        return false;
    do
    {
        if(item == element)
            return true;
    }
    while(getNext(item));
    return false;
}

```

```

//Summary: 检索一个数据
//
//Parameters:
//    element 目标数据
//Return: bool
//Detail:
template<class T>
bool linkedlist<T>::retrieve(T &element)
{
    if(!find(element))
        return false;
    element = current->data;
    return true;
}

```

```

//Summary: 替换一个数据
//
//Parameters:
//    newElement 目标数据

```

```

//Return: bool
//Detail: 将当前位置的数据替换
template<class T>
bool linkedlist<T>::replace(const T &newElement)
{
    if(current == NULL)
        return false;
    current->data = newElement;
    return true;
}

```

```

//Summary: 移除一个数据
//
//Parameters:
//    element 目标数据
//Return: bool
//Detail:
template<class T>
bool linkedlist<T>::remove(T &element)
{
    current = NULL;
    if(head == NULL)
        return false;
    node<T> *tmp = head;
    if(head->data == element)
    {
        element = head->data;
        head = tmp->next;
        delete tmp;
        return true;
    }
    while(tmp->next != NULL)
    {
        if(tmp->next->data == element)
        {
            element = tmp->next->data;
            node<T> *ptr = tmp->next;
            tmp->next = ptr->next;
            delete ptr;
            return true;
        }
        tmp = tmp->next;
    }
    return false;
}

```

```

}

//Summary: 判断是否为空
//
//Parameters:
//      None
//Return: bool
//Detail:
template<class T>
bool linkedlist<T>::isEmpty() const
{
    return head == NULL;
}

//Summary: 将链表清空
//
//Parameters:
//      None
//Return: None
//Detail:
template<class T>
void linkedlist<T>::makeEmpty()
{
    while(head != NULL)
    {
        current = head;
        head = head->next;
        delete current;
    }
    current = NULL;
}

//Summary: "="运算符重载
//
//Parameters:
//      rlist 右操作符
//Return: this
//Detail:
template<class T>
linkedlist<T>& linkedlist<T>::operator =(const linkedlist<T> &rlist)
{
    if(this == &rlist)
        return *this;
    makeEmpty();
}

```

```

        deepCopy(rlist);
        return *this;
    }

//Summary: 深拷贝函数
//
//Parameters:
//    original 原链表
//Return: None
//Detail:
template<class T>
void linkedlist<T>::deepCopy(const linkedlist<T> &original)
{
    head = current = NULL;
    if(original.head == NULL)
        return ;

    node<T> *copy = head = new node<T>;
    node<T> *origin = original.head;
    copy->data = origin->data;
    if(origin == original.current)
        current = copy;

    while(origin->next != NULL)
    {
        copy->next = new node<T>;
        origin = origin->next;
        copy = copy->next;
        copy->data = origin->data;
        if(origin == original.current)
            current = copy;
    }
    copy->next = NULL;
}

/*****
Filename: manager.cpp

Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: manager类的定义

*****/

```



```

#include <iostream>
#include "manager.h"
using namespace std;

//Summary: 输出经理的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 与employee类的函数相比，增加了岗位信息
void manager::printInfo()
{
    cout<<"输出个人信息: "<<endl;
    cout<<"岗位: 经理"<<endl;
    cout<<"编号: "<<getEmpNo()<<endl;
    cout<<"等级: "<<getGrade()<<endl;
    cout<<"月薪: "<<getAccumPay()<<endl<<endl;
}

//Summary: 无参构造函数
//
//Parameters:
//      None
//Return: None
//Detail: 调用上级构造函数，默认设置等级为
manager::manager():employee(1, FIXED_PAY_MANAGER)
{
}

//Summary: 有参构造函数
//
//Parameters:
//      inputGrade: 输入的雇员等级
//Return: None
//Detail: 调用上级构造函数，设置等级为输入等级，月薪为固定月薪
manager::manager(int inputGrade):employee(inputGrade, FIXED_PAY_MANAGER)
{
}

//Summary: 析构函数
//
//Parameters:

```

```

//      None
//Return: None
manager::~manager()
{

}

//Summary: 流输入操作符重载函数
//
//Parameters:
//      input: 输入流
//      m:      目标经理对象
//Return: istream& 返回输入流本身
//Detail: 重载函数首先给出输入提示，之后从流中输入目标的等级
//注:      未添加合法性判断，请勿输入非正数；月薪为固定，不能输入
istream& operator >> (istream& input, manager& m)
{
    cout<<"输入编号为"<<m.getEmpNo()<<" 的经理的等级: "<<endl;
    int tempGrade;
    input>>tempGrade;
    m.setGrade(tempGrade);

    return input;
}

//Summary: 流输出操作符重载函数
//
//Parameters:
//      output: 输出流
//      m:      目标经理对象
//Return: ostream& 返回输出流本身
ostream& operator << (ostream &output, manager &m)
{
    output<<"输出个人信息: "<<endl;
    output<<"岗位: 经理"<<endl;
    output<<"编号: "<<m.getEmpNo()<<endl;
    output<<"等级: "<<m.getGrade()<<endl;
    output<<"月薪: "<<m.getAccumPay()<<endl<<endl;
    return output;
}

/*****
Filename: salemanager.pp

```

Copyright: Tsinghua University

Author: Huaizhou Tao

Date:2014-10-02

Description: salemanager类的定义

```

/*****/
#include <iostream>
#include "salemanager.h"
using namespace std;

//Summary: 输出销售经理的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 与employee类的函数相比,增加了岗位信息与销售额
void salemanager::printInfo()
{
    cout<<"输出个人信息:"<<endl;
    cout<<"岗位: 销售经理"<<endl;
    cout<<"编号: "<<getEmpNo()<<endl;
    cout<<"等级: "<<getGrade()<<endl;
    cout<<"月薪: "<<getAccumPay()<<endl;
    cout<<"销售额: "<<getSales()<<endl;
    cout<<"提成率: "<<getCommissionRate()<<endl;
    //TODO: 列出该经理的下属销售员编号
    cout<<"下属销售员编号: "<<endl;
    int empNo;
    if(salesmanNoList.getFirst(empNo))
    {
        cout<<empNo<<" ";
        while(salesmanNoList.getNext(empNo))
            cout<<empNo<<" ";
        cout<<endl<<endl;
    }
    else
    {
        cout<<"无"<<endl<<endl;
    }
}

//Summary: 无参构造函数
//
```

```

//Parameters:
//      None
//Return: None
//Detail: 调用上级构造函数，默认设置等级为，
//      销售额为
salemanager::salemanager():manager(), salesman()
{
    setCommissionRate(RATE_SALEMANAGER);
    setAccumPay(FIXED_PAY_SALEMANAGER);
    //TODO: 初始化销售员编号链表
    salesmanNoList.makeEmpty();
}

//Summary: 有参构造函数
//
//Parameters:
//      inputGrade: 输入的雇员等级
//      inputSales: 该雇员的月销售额
//      inputSalesman: 下属的销售员编号
//      numberSalesman: 下属的销售员数目
//Return: None
//Detail: 调用上级构造函数，设置等级为输入等级，
//      工资由销售额决定
salemanager::salemanager(int inputGrade, int inputSales, linkedlist<int> &inputNoList)
:manager(inputGrade), salesman(inputGrade, inputSales, NULL)
{
    setCommissionRate(RATE_SALEMANAGER);
    setAccumPay(FIXED_PAY_SALEMANAGER + (int)(getSales() * getCommissionRate()));
    //TODO: 使用参数初始化链表
    salesmanNoList.makeEmpty();
    salesmanNoList = inputNoList;
}

//Summary: 析构函数
//
//Parameters:
//      None
//Return: None
salemanager::~salemanager()
{
    //TODO: 清理销售员列表
    salesmanNoList.makeEmpty();
}

```

```

//Summary: 更改销售额函数
//
//Parameters:
//      inputSales: 新增的销售额
//Return: None
//Detail: 输入可以为负数，代表负增长或销售员离开
void salemanager::changeSales(int inputSales)
{
    int sales = getSales();
    sales += inputSales;
    setSales(sales);
    setAccumPay(FIXED_PAY_SALEMANAGER + (int)(getSales() * getCommissionRate()));
}

//Summary: 增加销售员函数
//
//Parameters:
//      empNo: 新增的销售员编号
//Return: None
//Detail: 不能重复添加，但不会检查该编号是否存在，是否是一个销售员
bool salemanager::addSalesman(int empNo)
{
    //TODO: 检查与添加操作
    if(salesmanNoList.find(empNo))
        return false;
    salesmanNoList.insert_end(empNo);
    return true;
}

//Summary: 删除销售员函数
//
//Parameters:
//      empNo: 删除的销售员编号
//Return: None
//Detail: 如链表中不存在该编号，则返回false
bool salemanager::deleteSalesman(int empNo)
{
    return salesmanNoList.remove(empNo);
}

//Summary: 流输入操作符重载函数
//
//Parameters:
//      input: 输入流

```

```

//      sm:      目标销售经理对象
//Return: istream& 返回输入流本身
//Detail: 重载函数首先给出输入提示，之后从流中输入目标对象的等级
//注:      未添加合法性判断，请勿输入非正数；销售额由下属的总额决定，不能输入
istream& operator >> (istream& input, salemanager& sm)
{
    cout<<"输入编号为"<<sm.getEmpNo()<<" 的销售经理的等级: "<<endl;
    int tempGrade;
    input>>tempGrade;
    sm.setGrade(tempGrade);

    return input;
}

//Summary: 流输出操作符重载函数
//
//Parameters:
//      output: 输出流
//      sm:      目标销售经理对象
//Return: ostream& 返回输出流本身
ostream& operator << (ostream &output, salemanager &sm)
{
    output<<"输出个人信息: "<<endl;
    output<<"岗位: 销售经理"<<endl;
    output<<"编号: "<<sm.getEmpNo()<<endl;
    output<<"等级: "<<sm.getGrade()<<endl;
    output<<"月薪: "<<sm.getAccumPay()<<endl;
    output<<"销售额: "<<sm.getSales()<<endl;
    output<<"提成率: "<<sm.getCommissionRate()<<endl;
    //TODO: 列出该经理的下属销售员编号
    output<<"下属销售员编号: "<<endl;
    int empNo;
    if(sm.salesmanNoList.getFirst(empNo))
    {
        output<<empNo<<" ";
        while(sm.salesmanNoList.getNext(empNo))
            output<<empNo<<" ";
        output<<endl<<endl;
    }
    else
    {
        output<<"无"<<endl<<endl;
    }
    return output;
}

```

```

}

/*****
Filename: salesman.cpp

Copyright: Tsinghua University
Author: Huaizhou Tao
Date:2014-10-02
Description: salesman类的定义

*****/
#include <iostream>
#include "salesman.h"
#include "salemanager.h"
using namespace std;

//Summary: 输出销售员的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 与employee类的函数相比，增加了岗位信息与销售额
void salesman::printInfo()
{
    cout<<"输出个人信息: "<<endl;
    cout<<"岗位: 销售"<<endl;
    cout<<"编号: "<<getEmpNo()<<endl;
    cout<<"上司编号: "<<getBoss()->getEmpNo()<<endl;
    cout<<"等级: "<<getGrade()<<endl;
    cout<<"月薪: "<<getAccumPay()<<endl;
    cout<<"销售额: "<<getSales()<<endl;
    cout<<"提成率: "<<getCommissionRate()<<endl<<endl;
}

//Summary: 无参构造函数
//
//Parameters:
//      None
//Return: None
//Detail: 调用上级构造函数，默认设置等级为，
//      销售额与工资为
salesman::salesman():employee(1,0)
{
    sales = 0;
}

```

```

        commissionRate = RATE_SALESMAN;
        boss = NULL;
    }

//Summary: 有参构造函数
//
//Parameters:
//    inputGrade: 输入的雇员等级
//    inputSales: 该雇员的月销售额
//    inputBoss: 该雇员的上司
//Return: None
//Detail: 调用上级构造函数，设置等级为输入等级，
//    工资由销售额决定
salesman::salesman(int inputGrade, int inputSales, salemanager *
inputBoss):employee(inputGrade, 0)
{
    sales = inputSales;
    commissionRate = RATE_SALESMAN;
    setAccumPay((int)(sales * commissionRate));

    boss = inputBoss;
    if(boss != NULL)
    {
        boss->changeSales(sales);
        //TODO: boss添加该销售员的操作
        boss->addSalesman(getEmpNo());
    }
}

//Summary: 析构函数
//
//Parameters:
//    None
//Return: None
salesman::~salesman()
{
    if(boss !=NULL)
    {
        boss->changeSales(-sales);
        //TODO: boss删除该销售员的操作
        boss->deleteSalesman(getEmpNo());
    }
}

```



```

//Summary: 设置销售额函数
//
//Parameters:
//      inputSales: 新的销售额
//Return: None
//Detail: 因为涉及到上级的操作，不再使用内联函数
void salesman::setSales(int inputSales)
{
    if(boss != NULL)
    {
        boss->changeSales(inputSales - sales);
    }
    sales = inputSales;
    setAccumPay((int)(sales * commissionRate));
}

```

```

//Summary: 设置上级函数
//
//Parameters:
//      inputBoss: 新的上级
//Return: None
//Detail: 变更上级时，要同时变更上级的销售额
void salesman::setBoss(salemanager *inputBoss)
{
    if(boss != NULL)
    {
        boss->changeSales(-sales);
        //TODO: 原有boss删除该销售员的操作
        boss->deleteSalesman(getEmpNo());
    }
    if(inputBoss != NULL)
    {
        inputBoss->changeSales(sales);
        //TODO: 新boss添加该销售员的操作
        inputBoss->addSalesman(getEmpNo());
    }
    boss = inputBoss;
}

```

```

//Summary: 流输入操作符重载函数
//
//Parameters:
//      input: 输入流
//      s:      目标销售员对象

```

```

//Return: istream& 返回输入流本身
//Detail: 重载函数首先给出输入提示，之后从流中输入目标对象的等级与销售额
//注:      未添加合法性判断，请勿输入非正数；目前主函数的数据结构还未设计，
//          所以无法通过输入编号给销售员指定上级，该功能留待进一步完善
istream& operator >> (istream& input, salesman& s)
{
    cout<<"输入编号为"<<s.getEmpNo()<<" 的销售员的等级与销售额："<<endl;
    int tempGrade;
    input>>tempGrade;
    s.setGrade(tempGrade);
    input>>s.sales;
    s.setAccumPay((int)(s.sales * s.commissionRate));
    //TODO: 通过输入编号给销售员指定上级

    return input;
}

//Summary: 流输出操作符重载函数
//
//Parameters:
//      output: 输出流
//      s:      目标销售员对象
//Return: ostream& 返回输出流本身
ostream& operator << (ostream &output, salesman &s)
{
    output<<"输出个人信息："<<endl;
    output<<"岗位：销售"<<endl;
    output<<"编号："<<s.getEmpNo()<<endl;
    output<<"上司编号："<<s.getBoss()->getEmpNo()<<endl;
    output<<"等级："<<s.getGrade()<<endl;
    output<<"月薪："<<s.getAccumPay()<<endl;
    output<<"销售额："<<s.getSales()<<endl;
    output<<"提成率："<<s.getCommissionRate()<<endl<<endl;
    return output;
}

/*****
Filename:technician.cpp

```

Copyright: Tsinghua University

Author: Huaizhou Tao

Date:2014-10-02

Description: technician类的定义

```

*****/
#include <iostream>
#include "technician.h"
using namespace std;

//Summary: 输出技术员的基本信息
//
//Parameters:
//      None
//Return: None
//Detail: 与employee类的函数相比，增加了岗位信息与工作时间
void technician::printInfo()
{
    cout<<"输出个人信息: "<<endl;
    cout<<"岗位: 技术"<<endl;
    cout<<"编号: "<<getEmpNo()<<endl;
    cout<<"等级: "<<getGrade()<<endl;
    cout<<"月薪: "<<getAccumPay()<<endl;
    cout<<"工作时间: "<<getWorkHour()<<endl;
    cout<<"时薪: "<<getWage()<<endl<<endl;
}

//Summary: 无参构造函数
//
//Parameters:
//      None
//Return: None
//Detail: 调用上级构造函数，默认设置等级为，
//      工作时间与工资为
technician::technician():employee(1,0)
{
    workHour = 0;
    wage = WAGE_TECHNICIAN;
}

//Summary: 有参构造函数
//
//Parameters:
//      inputGrade: 输入的雇员等级
//      inputWorkHour: 该雇员的月工作时间
//Return: None
//Detail: 调用上级构造函数，设置等级为输入等级，
//      工资由工作时间决定
technician::technician(int inputGrade, int inputWorkHour):employee(inputGrade, 0)

```

```

{
    workHour = inputWorkHour;
    wage = WAGE_TECHNICIAN;
    setAccumPay(workHour * wage);
}

//Summary: 析构函数
//
//Parameters:
//    None
//Return: None
technician::~~technician()
{

}

//Summary: 流输入操作符重载函数
//
//Parameters:
//    input: 输入流
//    t:    目标技术员对象
//Return: istream& 返回输入流本身
//Detail: 重载函数首先给出输入提示，之后从流中输入目标对象的等级与工作时间
//注:    未添加合法性判断，请勿输入非正数；
istream& operator >> (istream& input, technician& t)
{
    cout<<"输入编号为"<<t.getEmpNo()<<" 的技术员的等级与工作时间: "<<endl;
    int tempGrade;
    input>>tempGrade;
    t.setGrade(tempGrade);
    input>>t.workHour;
    t.setAccumPay(t.workHour * t.wage);
    return input;
}

//Summary: 流输出操作符重载函数
//
//Parameters:
//    output: 输出流
//    t:    目标技术员对象
//Return: ostream& 返回输出流本身
ostream& operator << (ostream &output, technician &t)
{
    output<<"输出个人信息: "<<endl;

```

```

output<<"岗位: 技术"<<endl;
output<<"编号: "<<t.getEmpNo()<<endl;
output<<"等级: "<<t.getGrade()<<endl;
output<<"月薪: "<<t.getAccumPay()<<endl;
output<<"工作时间: "<<t.getWorkHour()<<endl;
output<<"时薪: "<<t.getWage()<<endl<<endl;
return output;
}

```

本章小结

继承技术是 c++主要的代码重用机制，也是 C++的三大特性之一。采用 C++提供了继承的机制，许多研发者就可以开发各种各样的类库，用户将这些类库作为基类去建立适合于自己的派生类，并在此基础上高效地开发应用程序。因此，面向对象程序设计最核心思想就是需要设计类的层次结构：即从最初的抽象类出发，每一层派生类的建立都逐步地向着目标的具体实现前进。另外，组合是也 C++的代码重用机制之一。

本章围绕 C++类的继承这一核心概念，重点介绍了派生类的 3 种继承方式和成员的 4 种访问属性。另外，还重点介绍了派生类在不同情况下的构造函数的定义方法。虚基类是解决多重继承中的同名二义性问题的主要方法。也是本章学习的重点。

课后习题

1. 分析本章“公司人事管理系统”程序代码，对不前一次代码，找出代码的不同之处，修改之处引入本章中所学的哪些知识点？
2. 类中成员有哪 3 种访问控制属性，分析为什么需要引入 protected 访问属性？
3. 如果一个派生类在基类的基础上要产生“变异”，即使得一个派生类与其基类有所不同，一般有哪些方法？
4. 要求定义一个基类 Person，它有 3 个 protected 的数据成员：姓名 name(char *类型)、性别 sex(char 类型)、年龄 age(int 类型)；一个构造函数用于对数据成员初始化。（1）创建 Person 类的公有派生类 Employee，增加两个数据成员 基本工资 basicSalary(int 类型) 请假天数 leaveDays (int 型)；为它定义初始化成员信息的构造函数，和显示数据成员信息的成员函数 show()。（2）创建 Employee 类的公有派生类 Manager；增加一个成员 业绩 performance(float 类型)；为它定义初始化成员信息的构造函数，和显示数据成员信息的成员函数 show()。（3）输入共 6 个数据，分别代表姓名、性别、年龄、基本工资、请假天数、业绩。每个数据之间用一个空格间隔。（4）输出如示例数据所示，共 5 行，分别代表姓名、年龄、性别、基本工资、请假天数、业绩。
5. 设计一个 People(人员)类，具有的属性如下：姓名 char name[11]、编号 char number[7]、

性别 char sex[3]、生日 birthday、身份证号 char id[16]。其中“出生日期”声明为一个“日期”类内嵌子对象。用成员函数实现对人员信息的录入和显示。要求包括：构造函数和析构函数、拷贝构造函数、内联成员函数、运算符重载等。在测试程序中声明 people 类的对象数组，录入数据并显示。

6. 在第 5 题的基础上，从 People(人员)类派生出 student(学生)类，添加属性：班号 char classNo[7]；从 People(人员)类派生出 teacher(教师)类，添加属性：职务 char principalship[11]、部门 char department[21]；从 student 类中派生出 graduate(研究生)类、添加属性：专业 char subject[21]、导师 teacher advisor；从 teacher 类和 graduate 类派生出 TA(助教)类，注意虚基类的使用。要求编制一能管理上述四类人员的测试程序，能实现数据项录入、显示、删除操作。
7. 在第 6 题的基础上，补充各类（People、student、graduate、teacher 和 TA）的构造函数和析构函数。
 - (1) 编程测试和分析这些构造函数和析构函数的执行顺序
 - (2) 如果不采用虚基类，重新编程实现第 5 次作业要求。体会会有什么差异？
8. 分析下列程序，运行程序的执行结果

```
#include <iostream.h>

class Base
{
private:
    int b_number;

public:
    Base( ) {}
    Base(int i) : b_number (i) { }
    int get_number( ) {return b_number;}
    void print( ) {cout << b_number << endl;}
};

class Derived : public Base
{
private:
    int d_number;

public:
    Derived( int i, int j ) : Base(i), d_number(j) { };
    void print( )
    {
        cout << get_number( ) << " ";
        cout << d_number << endl;
    }
};

int main( )
{
```

```

        Base a(2);
        Derived b(3, 4);
        cout << "a is ";
        cout << "b is ";
        b.print();
        cout << "base part of b is ";
        b.Base::print();
        return 0;
}

```

(1) 将“class Derived : public Base”改为“class Derived : private Base”，再调试程序，看看有什么不一样。

(2) 将“class Derived : public Base”改为“class Derived : protected Base”，再调试程序，看看有什么不一样。

9. 分析下列程序运行时的构造函数和析构函数执行顺序

```

#include<iostream>
using namespace std;
class B1{
public:
    B1(int i){ cout<<" constructing B1 "<<i<<endl; }
    ~B1() { cout<<" destructing B1 "<<endl; }
};
class B2 {
public:
    B2() { cout<<" constructing B3 *" <<endl; }
    ~B2() { cout<<" destructing B3" <<endl; }
};
class C:public B2, virtual public B1 {
    int j;
public:
    C(int a,int b,int c):B1(a),memberB1(b) ,j(c) {}
private:
    B1 memberB1;
    B2 memberB2;
};
int main() {
    C obj(1,2,3);
}

```

10. 分析下列程序的执行结果

```

#include<iostream>
using namespace std;
class B{
public:

```

```
        void f1() {cout<<"B::f1"<<endl;}
};
class D:public B{
public:
    void f1() {cout<<"D::f1"<<endl;}
};
void f(B& rb) {
    rb.f1();
}
int main( )
{
    D d;
    B b,&rb1=b,&rb2=d;
    f(rb1);
    f(rb2);
    return 0;
}
```